

Министерство науки и высшего образования РФ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Новосибирский государственный технический университет»
Кафедра Автоматизированных систем управления

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине «**Технологии и методы программирования**»

«ОРГАНИЗАЦИЯ МНОГОСВЯЗНЫХ СТРУКТУР»

Выполнили:
Студенты группы АВТ-414
Кривошапов С.В.
Зарубин Р.Д.
Ереза М.Д.

Новосибирск 2026

Оглавление

1. Текст задания	3
2. Графическое описание структур, используемых в процедурах манипуляции со списком ...	6
3. Исходный код	6
4. Скриншоты дампа файлов.....	91

1. Текст задания

Задание 1

Разработать консольное приложение для работы с данными, логическая организация которых имеет следующий вид (рис.2).

Тестирующая программа (консольное приложение) при запуске выводит подсказку PS> и ожидает команду.

Список команд включает:

Create *имя файла*(максимальная длина имени компонента[, имя файла спецификаций]) — если файл существует и сигнатура соответствует заданию, команда требует подтверждения на перезапись файла. При положительном ответе, файлы очищаются, после чего создаются все необходимые структуры в памяти и файлах на диске. После успешного выполнения команды файлы считаются открытыми для работы. Если сигнатура файла отсутствует или не соответствует заданию, команда вызывает ошибку. Расширение имени файла для списка компонентов — «.prd», а для файла спецификаций — «.prs».

Open *имя файла* — открывает указанный файл и связанные с ним файлы в режиме rw, создает все необходимые структуры в памяти. Если сигнатура файла отсутствует или не соответствует заданию, команда вызывает ошибку.

Input (*имя компонента, тип*) — включает компонент в список. *тип* — одно из следующего: Изделие, Узел, Деталь.

Input (*имя компонента/имя комплектующего*) — включает комплектующее в спецификацию компонента. Имя комплектующего должно быть в списке, в противном случае и для детали эта команда вызывает ошибку.

Delete (*имя компонента*) — логически удаляет запись с именем компонента из списка, устанавливая бит удаления в -1. Если на компонент имеются ссылки в спецификациях других компонент, эта команда вызывает ошибку.

Delete (*имя компонента/имя комплектующего*) — логически удаляет комплектующее из спецификации компонента, устанавливая бит удаления в -1. Для детали эта команда вызывает ошибку.

Restore (*имя компонента*) — снимает бит удаления (присваивает значение 0) со всех записей, относящихся к заданному компоненту и ранее помеченных на удаление, а также

восстанавливает алфавитный порядок, который мог быть нарушен из-за добавления новых записей.

Restore (*) — снимает бит удаления (присваивает значение 0) со всех записей, ранее помеченных на удаление, и восстанавливает алфавитный порядок, который мог быть нарушен из-за добавления новых записей.

Truncate — физически удаляет из списков записи, бит удаления которых установлен в -1, и перераспределяет записи списков таким образом, что все они становятся смежными, а свободная область располагается в конце файлов. Корректирует указатель на свободную область файла;

Print (имя компонента) — выводит на экран состав компонента (спецификацию) в виде (для детали эта команда вызывает ошибку):

```
Компонент
|
| Узел
| |
| | Деталь
|
| Деталь
```

Print (*) — выводит на экран построчно список компонентов в формате

Наименование	Тип
--------------	-----

Help [имя файла] — выводит на экран или в указанный файл список команд.

После выполнения этих команд или сообщения об ошибке выводится подсказка PS> и ожидается следующая команда.

Exit — закрывает все файлы и завершает программу. Файлы при завершении программы не уничтожаются. Они уничтожаются вручную после просмотра дампа файлов при защите.

Задание 2

Разработать оконное приложение для работы с данными, логическая организация которых имеет следующий вид (рис.2).

Интерфейс программы показан на рисунках 3 - 5.

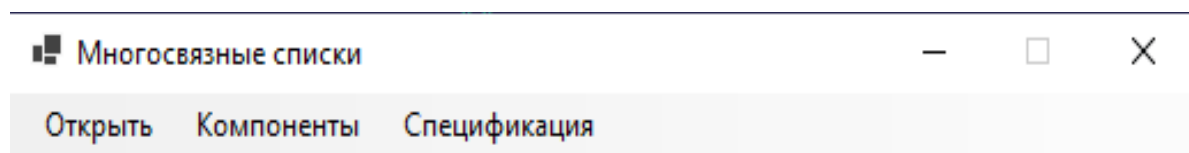


Рисунок 3 — Меню главной формы

Наименование	Тип
Изделие1	Изделие
Узел1	Узел

Рисунок 4 — Форма со списком компонентов

Спецификация

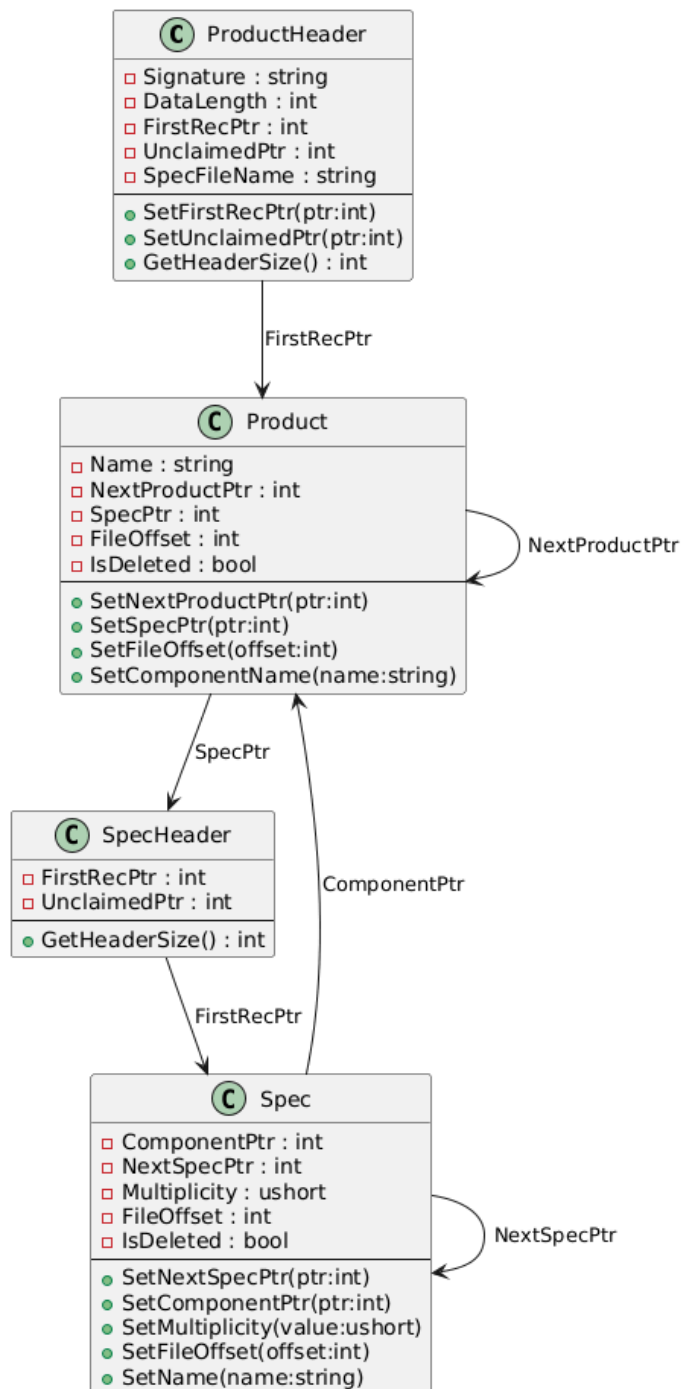
Найти

- Изделие1
 - Узел1
 - Деталь1
 - Узел2

Добавить
Изменить
Удалить

Рисунок 5 — Форма для создания спецификации

2. Графическое описание структур, используемых в процедурах манипуляции со списком



3. Исходный код. Листинги всех файлов.

FileStructure.cs

```
using System;

namespace ConsoleApp2.Data.Abstractions
{
    public static class FileStructure
    {
        public const string DEFAULT_SIGNATURE = "PS";
        public const int SIGNATURE_SIZE = 2;
        public const int DATA_LENGTH_SIZE = 2;
        public const int POINTER_SIZE = 4;
        public const int SPEC_FILENAME_SIZE = 16;
        public const int DELBIT_SIZE = 1;
        public const int MULTIPLICITY_SIZE = 2;
        public const int TYPE_SIZE = 1;
    }
}
```

IFSManager.cs

```
using System;
using System.Collections.Generic;
using System.Text;
using System.IO;
using ConsoleApp2.Entities;

namespace ConsoleApp2.Data.Abstractions
{
    public interface IFSManager
    {
        FileStream GetStream();
        void WriteHeader(ProductHeader header);
        ProductHeader ReadHeader();
        void Seek(long offset);
        long GetPosition();
        void Close();
        void CreateFile();
        void OpenFile();
    }
}
```

ILLManager.cs

```
using System;
using System.Collections.Generic;
using System.Text;
using ConsoleApp2.Entities;
```

```

namespace ConsoleApp2.Data.Abstractions
{
    public interface ILLManager
    {
        void Initialize(ProductHeader header);
        void AddProduct();
        void AddSpec();
        void Delete(int offset);
        void RestoreProduct(Product product);
        void RestoreSpec();
        void RestoreAll();
        void Truncate();
        Product FindProduct(string name);
        Spec FindSpec(string prodName, string specName);
        void SortAlphabetically();
        void LoadFromFile();
        void Update();
    }
}

```

IRepository.cs

```

using System;
using System.Collections.Generic;
using System.Text;
using ConsoleApp2.Entities;

namespace ConsoleApp2.Data.Abstractions
{
    public interface IRepository
    {
        void Create(string filePath, short dataLength, string specFileName);
        void Open(string filePath);
        void Close();
        void Add(Product product);
        void Delete(string productName);
        void Restore(string productName);
        void RestoreAll();
        void Truncate();
        Product Find(string productName);
        Product FindIncludeDeleted(string productName);
        IEnumerable<Product> GetAll();
    }
}

```

ISerializer.cs

```

using System;
using System.Collections.Generic;

```

```

using System.Text;
using ConsoleApp2.Entities;

namespace ConsoleApp2.Data.Abstractions
{
    public interface IRepository
    {
        void Create(string filePath, short dataLength, string specFileName);
        void Open(string filePath);
        void Close();
        void Add(Product product);
        void Delete(string productName);
        void Restore(string productName);
        void RestoreAll();
        void Truncate();
        Product Find(string productName);
        Product FindIncludeDeleted(string productName);
        IEnumerable<Product> GetAll();
    }
}

```

BaseFSManager.cs

```

using ConsoleApp2.Entities;
using System;
using System.Collections.Generic;
using System.Text;
using System.IO;

namespace ConsoleApp2.Data
{
    public class BaseFSManager
    {
        protected string _filePath;
        protected FileStream _fileStream;

        public BaseFSManager(string filePath)
        {
            _filePath = filePath;
        }

        public FileStream GetStream() => _fileStream;

        public void Seek(long offset)
        {
            if (_fileStream == null)
                throw new InvalidOperationException("File not opened.");
            _fileStream.Seek(offset, SeekOrigin.Begin);
        }
    }
}

```

```

public long GetPosition()
{
    return _fileStream?.Position ?? -1;
}

public void Close()
{
    _fileStream?.Close();
    _fileStream?.Dispose();
    _fileStream = null;
}

public void CreateFile()
{
    if (_fileStream != null) _fileStream.Close();

    if (File.Exists(_filePath))
    {
        File.Delete(_filePath);
    }

    _fileStream = new FileStream(_filePath, FileMode.CreateNew,
FileAccess.ReadWrite);
}

public void OpenFile()
{
    if (_fileStream != null) _fileStream.Close();

    if (!File.Exists(_filePath))
        throw new FileNotFoundException($"File not found at {_filePath}.");

    _fileStream = new FileStream(_filePath, FileMode.Open, FileAccess.ReadWrite);
}

public FileStream CreateTempFile()
{
    string tempFilePath = _filePath + ".tmp";

    if (File.Exists(tempFilePath))
        File.Delete(tempFilePath);

    return new FileStream(tempFilePath, FileMode.CreateNew, FileAccess.ReadWrite);
}

public string GetTempFilePath()
{
    return _filePath + ".tmp";
}

public void ReplaceWithTempFile()

```

```

    {
        string tempFilePath = GetTempFilePath();

        if (!File.Exists(tempFilePath))
            throw new FileNotFoundException($"Temp file not found at {tempFilePath}.");

        Close();

        if (File.Exists(_filePath))
            File.Delete(_filePath);

        File.Move(tempFilePath, _filePath);

        OpenFile();
    }
}

```

LLManager.cs

```

using ConsoleApp2.Entities;
using ConsoleApp2.Data.Abstractions;
using System;
using System.Collections.Generic;
using System.Text;
using System.Linq;
using System.IO;

namespace ConsoleApp2.Data
{
    public class LLManager
    {
        private ProductFSManager _productFsManager;
        private SpecFSManager _specFsManager;
        private ProductSerializer _productSerializer;
        private SpecSerializer _specSerializer;
        private ProductHeader _productHeader;
        private Dictionary<string, Product> _productCache;
        private Dictionary<int, SpecHeader> _specHeaderCache;
        private Dictionary<int, List<Spec>> _specListCache;

        public LLManager(ProductFSManager productFsManager, SpecFSManager
specFsManager,
            ProductSerializer productSerializer, SpecSerializer specSerializer)
        {
            _productFsManager = productFsManager ?? throw new
ArgumentNullException(nameof(productFsManager));
            _specFsManager = specFsManager ?? throw new
ArgumentNullException(nameof(specFsManager));

```

```

        _productSerializer = productSerializer ?? throw new
ArgumentNullException(nameof(productSerializer));
        _specSerializer = specSerializer ?? throw new
ArgumentNullException(nameof(specSerializer));
        _productCache = new Dictionary<string, Product>();
        _specHeaderCache = new Dictionary<int, SpecHeader>();
        _specListCache = new Dictionary<int, List<Spec>>();
    }

    public void Initialize(ProductHeader productHeader)
    {
        _productHeader = productHeader ?? throw new
ArgumentNullException(nameof(productHeader), "Product header is null.");
        _productCache.Clear();
        _specHeaderCache.Clear();
        _specListCache.Clear();
    }

    public void AddProduct(Product product)
    {
        if (product == null)
            throw new ArgumentNullException(nameof(product), "Product is null.");

        if (_productCache.ContainsKey(product.Name))
            throw new InvalidOperationException("Component already exists.");

        int offset = (int)_productFsManager.GetStream().Length;
        product.FileOffset = offset;

        _productFsManager.Seek(offset);
        using (var writer = new BinaryWriter(_productFsManager.GetStream(),
Encoding.UTF8, leaveOpen: true))
        {
            _productSerializer.WriteToFile(product, writer);
        }
        _productFsManager.GetStream().Flush();

        _productCache[product.Name] = product;

        _specHeaderCache[product.FileOffset] = new SpecHeader(-1,
SpecHeader.GetHeaderSize());
        _specListCache[product.FileOffset] = new List<Spec>();
        SortAlphabetically();
    }

    public void AddSpec(int productOffset, Spec spec)
    {
        if (spec == null)
            throw new ArgumentNullException(nameof(spec), "Spec is null.");

        if (! _specListCache.ContainsKey(productOffset))

```

```

        _specListCache[productOffset] = new List<Spec>();

        int offset = (int)_specFsManager.GetStream().Length;
        spec.FileOffset = offset;

        _specFsManager.Seek(offset);
        using (var writer = new BinaryWriter(_specFsManager.GetStream(), Encoding.UTF8,
leaveOpen: true))
        {
            _specSerializer.WriteToFile(spec, writer);
        }
        _specFsManager.GetStream().Flush();

        if (!_specHeaderCache.ContainsKey(productOffset))
        {
            var product = _productCache.Values.FirstOrDefault(p => p.FileOffset ==
productOffset);
            if (product != null && product.SpecPtr != -1)
            {
                _specHeaderCache[productOffset] = new SpecHeader(-1, product.SpecPtr +
SpecHeader.GetHeaderSize());
            }
            else if (product != null)
            {
                _specHeaderCache[productOffset] = new SpecHeader(-1,
SpecHeader.GetHeaderSize());
            }
        }

        var specHeader = _specHeaderCache[productOffset];
        if (specHeader != null && specHeader.FirstRecPtr == -1)
        {
            specHeader.FirstRecPtr = offset;
        }

        var comp = _productCache.Values.FirstOrDefault(p => p.FileOffset ==
spec.ComponentPtr);
        if (comp != null)
            spec.SetName(comp.Name);

        _specListCache[productOffset].Add(spec);

        SortAlphabetically();
    }

    public void Delete(int offset)
    {
        var product = _productCache.Values.FirstOrDefault(p => p.FileOffset == offset);
        if (product != null)
        {
            product.MarkAsDeleted();

```

```

        UpdateProduct(product);
    }
}

public void DeleteSpec(int productOffset, int specOffset)
{
    if (_specListCache.ContainsKey(productOffset))
    {
        var spec = _specListCache[productOffset].FirstOrDefault(s => s.FileOffset ==
specOffset);
        if (spec != null)
        {
            spec.MarkAsDeleted();
            UpdateSpec(spec);
        }
    }
}

public void Restore(string productName)
{
    if (_productCache.TryGetValue(productName, out var product))
    {
        product.Restore();
        UpdateProduct(product);

        if (_specListCache.ContainsKey(product.FileOffset))
        {
            foreach (var spec in _specListCache[product.FileOffset])
            {
                if (spec.IsDeleted)
                {
                    spec.Restore();
                    UpdateSpec(spec);
                }
            }
        }
        SortAlphabetically();
    }
}

public void RestoreAllProducts()
{
    foreach (var product in _productCache.Values)
    {
        if (product.IsDeleted)
        {
            product.Restore();
            UpdateProduct(product);
        }
    }
    SortAlphabetically();
}

```

```

    }

    public void RestoreAllSpecs()
    {
        foreach (var specList in _specListCache.Values)
        {
            foreach (var spec in specList)
            {
                if (spec.IsDeleted)
                {
                    spec.Restore();
                    UpdateSpec(spec);
                }
            }
        }

        foreach (var product in _productCache.Values)
        {
            if (_specListCache.ContainsKey(product.FileOffset))
            {
                var specs = _specListCache[product.FileOffset].Where(s =>
!s.IsDeleted).ToList();
                if (specs.Count > 0)
                {
                    _specHeaderCache[product.FileOffset].FirstRecPtr = specs[0].FileOffset;
                }
            }
        }

        SortAlphabetically();
    }

    public void Truncate()
    {
        var activeProducts = _productCache.Values
            .Where(p => !p.IsDeleted)
            .OrderBy(p => p.FileOffset)
            .ToList();

        if (activeProducts.Count == 0)
        {
            _productCache.Clear();
            _specHeaderCache.Clear();
            _specListCache.Clear();

            _productFsManager.CreateFile();
            _specFsManager.CreateFile();

            _productHeader.SetFirstRecPtr(-1);
            _productHeader.SetUnclaimedPtr(ProductHeader.GetHeaderSize());
            _productFsManager.WriteHeader(_productHeader);
        }
    }

```

```

        return;
    }

    int productHeaderSize = ProductHeader.GetHeaderSize();
    int productEntitySize = _productSerializer.GetEntitySize();
    int specEntitySize = _specSerializer.GetEntitySize();
    var oldProductOffsets = activeProducts.Select(p => p.FileOffset).ToList();
    var offsetMapping = new Dictionary<int, int>();

    var tempProductStream = _productFsManager.CreateTempFile();
    var tempSpecStream = _specFsManager.CreateTempFile();

    try
    {
        using (var writer = new BinaryWriter(tempProductStream, Encoding.UTF8,
leaveOpen: true))
        {
            _productHeader.SetFirstRecPtr((activeProducts.Count > 0) ? productHeaderSize
: -1);

writer.Write(Encoding.ASCII.GetBytes(_productHeader.Signature.PadRight(2).Substring(0,
2)));

            writer.Write(_productHeader.DataLength);
            writer.Write(_productHeader.FirstRecPtr);
            writer.Write(0);

            if (!string.IsNullOrEmpty(_productHeader.SpecFileName))
            {
                char[] specBuffer = new char[16];
                Array.Copy(_productHeader.SpecFileName.ToCharArray(), specBuffer,
                    Math.Min(_productHeader.SpecFileName.Length, 16));
                writer.Write(specBuffer);
            }

            for (int i = 0; i < activeProducts.Count; i++)
            {
                int newOffset = (int)tempProductStream.Position;
                offsetMapping[oldProductOffsets[i]] = newOffset;
                activeProducts[i].SetFileOffset(newOffset);

                _productSerializer.WriteToFile(activeProducts[i], writer);

                int newUnclaimedPtr = (int)tempProductStream.Position;
                _productHeader.SetUnclaimedPtr(newUnclaimedPtr);
            }
        }

        using (var writer = new BinaryWriter(tempSpecStream, Encoding.UTF8,
leaveOpen: true))
        {

```

```

for (int i = 0; i < activeProducts.Count; i++)
{
    int oldProductOffset = oldProductOffsets[i];

    if (_specListCache.TryGetValue(oldProductOffset, out var specs))
    {
        var activeSpecs = specs
            .Where(s => !s.IsDeleted)
            .OrderBy(s => s.Name)
            .ToList();

        if (activeSpecs.Count > 0)
        {
            tempSpecStream.Seek(0, SeekOrigin.End);
            int headerPos = (int)tempSpecStream.Position;
            activeProducts[i].SetSpecPtr(headerPos);

            int firstSpecPos = headerPos + SpecHeader.GetHeaderSize();
            writer.Write(firstSpecPos);
            writer.Write(firstSpecPos);
            writer.Flush();

            int unclaimedPtrPos = headerPos + FileStructure.POINTER_SIZE;

            int lastSpecOffset = firstSpecPos;
            for (int j = 0; j < activeSpecs.Count; j++)
            {
                int newSpecOffset = (int)tempSpecStream.Position;
                activeSpecs[j].SetFileOffset(newSpecOffset);

                int oldComponentPtr = activeSpecs[j].ComponentPtr;
                if (offsetMapping.ContainsKey(oldComponentPtr))
                {
                    activeSpecs[j].SetComponentPtr(offsetMapping[oldComponentPtr]);
                }

                activeSpecs[j].SetNextSpecPtr((j < activeSpecs.Count - 1)
                    ? newSpecOffset + specEntitySize
                    : -1);

                _specSerializer.WriteToFile(activeSpecs[j], writer);
                writer.Flush();

                lastSpecOffset = (int)tempSpecStream.Position;
                _productHeader.SetUnclaimedPtr(lastSpecOffset);
            }

            tempSpecStream.Seek(unclaimedPtrPos, SeekOrigin.Begin);
            writer.Write(lastSpecOffset);
            writer.Flush();

```

```

        tempSpecStream.Seek(0, SeekOrigin.End);
    }
    else
    {
        activeProducts[i].SetSpecPtr(-1);
    }
}
else
{
    activeProducts[i].SetSpecPtr(-1);
}
}
}

for (int i = 0; i < activeProducts.Count; i++)
{
    activeProducts[i].SetNextProductPtr((i < activeProducts.Count - 1)
        ? activeProducts[i].FileOffset + productEntitySize
        : -1);
}

tempProductStream.Seek(0, SeekOrigin.Begin);
using (var writer = new BinaryWriter(tempProductStream, Encoding.UTF8,
leaveOpen: true))
{
    writer.Write(Encoding.ASCII.GetBytes(_productHeader.Signature.PadRight(2).Substring(0,
2)));
    writer.Write(_productHeader.DataLength);
    writer.Write(_productHeader.FirstRecPtr);
    writer.Write(_productHeader.UnclaimedPtr);

    if (!string.IsNullOrEmpty(_productHeader.SpecFileName))
    {
        char[] specBuffer = new char[16];
        Array.Copy(_productHeader.SpecFileName.ToCharArray(), specBuffer,
            Math.Min(_productHeader.SpecFileName.Length, 16));
        writer.Write(specBuffer);
    }
}

for (int i = 0; i < activeProducts.Count; i++)
{
    tempProductStream.Seek(activeProducts[i].FileOffset, SeekOrigin.Begin);
    using (var writer = new BinaryWriter(tempProductStream, Encoding.UTF8,
leaveOpen: true))
    {
        _productSerializer.WriteToFile(activeProducts[i], writer);
    }
}

```

```

        tempProductStream.Flush();
        tempSpecStream.Flush();
    }
    catch
    {
        if (File.Exists(_productFsManager.GetTempFilePath()))
            File.Delete(_productFsManager.GetTempFilePath());
        if (File.Exists(_specFsManager.GetTempFilePath()))
            File.Delete(_specFsManager.GetTempFilePath());
        throw;
    }
    finally
    {
        tempProductStream?.Dispose();
        tempSpecStream?.Dispose();
    }

    _productFsManager.ReplaceWithTempFile();
    _specFsManager.ReplaceWithTempFile();

    _productCache = activeProducts.ToDictionary(p => p.Name, p => p);

    var newSpecListCache = new Dictionary<int, List<Spec>>>();
    for (int i = 0; i < activeProducts.Count; i++)
    {
        int oldProductOffset = oldProductOffsets[i];
        if (_specListCache.TryGetValue(oldProductOffset, out var specs))
        {
            var activeSpecs = specs.Where(s => !s.IsDeleted).ToList();
            if (activeSpecs.Count > 0)
            {
                newSpecListCache[activeProducts[i].FileOffset] = activeSpecs;
            }
        }
    }
    _specListCache = newSpecListCache;

    _specHeaderCache.Clear();
    foreach (var product in activeProducts)
    {
        if (_specListCache.ContainsKey(product.FileOffset))
        {
            var specs = _specListCache[product.FileOffset];
            int firstSpecPtr = specs.Count > 0 ? specs[0].FileOffset : -1;
            int unclaimedPtr = specs.Count > 0 ? specs[specs.Count - 1].FileOffset +
specEntitySize : SpecHeader.GetHeaderSize();
            _specHeaderCache[product.FileOffset] = new SpecHeader(firstSpecPtr,
unclaimedPtr);
        }
    }
}

```

```

public Product FindByName(string name)
{
    if (_productCache.TryGetValue(name, out var product) && !product.IsDeleted)
        return product;
    return null;
}

public Product FindByNameIncludeDeleted(string name)
{
    if (_productCache.TryGetValue(name, out var product))
        return product;
    return null;
}

public Product FindByOffset(int offset)
{
    return _productCache.Values.FirstOrDefault(p => p.FileOffset == offset &&
!p.IsDeleted);
}

public Spec FindSpecByOffset(int specOffset)
{
    foreach (var specList in _specListCache.Values)
    {
        var spec = specList.FirstOrDefault(s => s.FileOffset == specOffset && !s.IsDeleted);
        if (spec != null)
            return spec;
    }
    return null;
}

public IEnumerable<Product> GetAllProducts()
{
    return _productCache.Values.Where(p => !p.IsDeleted);
}

public Product GetLastProduct()
{
    var allProducts = GetAllProducts().ToList();
    if (allProducts.Count == 0)
        return null;

    return allProducts.Last();
}

public void UpdateProduct(Product product)
{
    if (product == null)
        throw new ArgumentNullException(nameof(product), "Product is null.");
}

```

```

        if (product.FileOffset == -1)
            throw new InvalidOperationException("Product must have valid offset.");

        _productFsManager.Seek(product.FileOffset);
        using (var writer = new BinaryWriter(_productFsManager.GetStream(),
Encoding.UTF8, leaveOpen: true))
        {
            _productSerializer.WriteToFile(product, writer);
        }
        _productFsManager.GetStream().Flush();
    }

    public void UpdateSpec(Spec spec)
    {
        if (spec == null)
            throw new ArgumentNullException(nameof(spec), "Spec is null.");

        if (spec.FileOffset == -1)
            throw new InvalidOperationException("Spec must have valid offset.");

        _specFsManager.Seek(spec.FileOffset);
        using (var writer = new BinaryWriter(_specFsManager.GetStream(), Encoding.UTF8,
leaveOpen: true))
        {
            _specSerializer.WriteToFile(spec, writer);
        }
        _specFsManager.GetStream().Flush();
    }

    public void LoadFromFile()
    {
        _productCache.Clear();
        _specHeaderCache.Clear();
        _specListCache.Clear();

        int currentOffset = _productHeader.FirstRecPtr;
        while (currentOffset != -1 && currentOffset != 1)
        {
            _productFsManager.Seek(currentOffset);
            using (var reader = new BinaryReader(_productFsManager.GetStream(),
Encoding.UTF8, leaveOpen: true))
            {
                var product = _productSerializer.ReadFromFile(reader);
                product.FileOffset = currentOffset;
                _productCache[product.Name] = product;

                if (!_specListCache.ContainsKey(product.FileOffset))
                {
                    _specListCache[product.FileOffset] = new List<Spec>();
                }
            }
        }
    }

```

```

        currentOffset = product.NextProductPtr;
    }
}

foreach (var product in _productCache.Values.ToList())
{
    if (product.SpecPtr != -1 && product.SpecPtr != 1)
    {
        _specFsManager.Seek(product.SpecPtr);
        using (var reader = new BinaryReader(_specFsManager.GetStream(),
Encoding.UTF8, leaveOpen: true))
        {
            var specHeader = new SpecHeader(reader.ReadInt32(), reader.ReadInt32());
            _specHeaderCache[product.FileOffset] = specHeader;

            int specOffset = specHeader.FirstRecPtr;
            while (specOffset != -1 && specOffset != 1)
            {
                _specFsManager.Seek(specOffset);
                using (var reader2 = new BinaryReader(_specFsManager.GetStream(),
Encoding.UTF8, leaveOpen: true))
                {
                    var spec = _specSerializer.ReadFromFile(reader2);
                    spec.FileOffset = specOffset;

                    if (!_specListCache.ContainsKey(product.FileOffset))
                    {
                        _specListCache[product.FileOffset] = new List<Spec>();
                    }

                    var comp = _productCache.Values.FirstOrDefault(p => p.FileOffset ==
spec.ComponentPtr);
                    if (comp != null)
                        spec.SetName(comp.Name);

                    _specListCache[product.FileOffset].Add(spec);
                    specOffset = spec.NextSpecPtr;
                }
            }
        }
    }
    else if (!_specListCache.ContainsKey(product.FileOffset))
    {
        _specListCache[product.FileOffset] = new List<Spec>();
    }
}

public void SortAlphabetically()
{
    var sortedProducts = _productCache.Values

```

```

        .Where(p => !p.IsDeleted)
        .OrderBy(p => p.Name)
        .ToList();

    if (sortedProducts.Count == 0)
    {
        _productCache.Clear();
        _specHeaderCache.Clear();
        _specListCache.Clear();

        _productFsManager.CreateFile();
        _specFsManager.CreateFile();

        _productHeader.SetFirstRecPtr(-1);
        _productHeader.SetUnclaimedPtr(ProductHeader.GetHeaderSize());
        _productFsManager.WriteHeader(_productHeader);

        return;
    }

    var oldProductOffsets = sortedProducts.Select(p => p.FileOffset).ToList();
    var offsetMapping = new Dictionary<int, int>();

    int productHeaderSize = ProductHeader.GetHeaderSize();
    int productEntitySize = _productSerializer.GetEntitySize();
    int specEntitySize = _specSerializer.GetEntitySize();

    var tempProductStream = _productFsManager.CreateTempFile();
    var tempSpecStream = _specFsManager.CreateTempFile();

    try
    {
        using (var writer = new BinaryWriter(tempProductStream, Encoding.UTF8,
leaveOpen: true))
        {
            _productHeader.SetFirstRecPtr((sortedProducts.Count > 0) ?
productHeaderSize : -1);

writer.Write(Encoding.ASCII.GetBytes(_productHeader.Signature.PadRight(2).Substring(0,
2)));

            writer.Write(_productHeader.DataLength);
            writer.Write(_productHeader.FirstRecPtr);
            writer.Write(0);

            if (!string.IsNullOrEmpty(_productHeader.SpecFileName))
            {
                char[] specBuffer = new char[16];
                Array.Copy(_productHeader.SpecFileName.ToCharArray(), specBuffer,
                    Math.Min(_productHeader.SpecFileName.Length, 16));
                writer.Write(specBuffer);
            }
        }
    }

```

```

    }

    for (int i = 0; i < sortedProducts.Count; i++)
    {
        int newOffset = (int)tempProductStream.Position;
        offsetMapping[oldProductOffsets[i]] = newOffset;
        sortedProducts[i].SetFileOffset(newOffset);

        _productSerializer.WriteToFile(sortedProducts[i], writer);
        int newUnclaimedPtr = (int)tempProductStream.Position;
        _productHeader.SetUnclaimedPtr(newUnclaimedPtr);
    }
}

using (var writer = new BinaryWriter(tempSpecStream, Encoding.UTF8,
leaveOpen: true))
{
    for (int i = 0; i < sortedProducts.Count; i++)
    {
        int oldProductOffset = oldProductOffsets[i];

        if (_specListCache.TryGetValue(oldProductOffset, out var specs))
        {
            var activeSpecs = specs
                .Where(s => !s.IsDeleted)
                .OrderBy(s => s.Name)
                .ToList();

            if (activeSpecs.Count > 0)
            {
                tempSpecStream.Seek(0, SeekOrigin.End);
                int headerPos = (int)tempSpecStream.Position;
                sortedProducts[i].SetSpecPtr(headerPos);

                int firstSpecPos = headerPos + SpecHeader.GetHeaderSize();
                writer.Write(firstSpecPos);
                writer.Write(firstSpecPos);
                writer.Flush();

                int unclaimedPtrPos = headerPos + FileStructure.POINTER_SIZE;

                int lastSpecOffset = firstSpecPos;
                for (int j = 0; j < activeSpecs.Count; j++)
                {
                    int newSpecOffset = (int)tempSpecStream.Position;
                    activeSpecs[j].SetFileOffset(newSpecOffset);

                    int oldComponentPtr = activeSpecs[j].ComponentPtr;
                    if (offsetMapping.ContainsKey(oldComponentPtr))
                    {
                        activeSpecs[j].SetComponentPtr(offsetMapping[oldComponentPtr]);
                    }
                }
            }
        }
    }
}

```

```

    }

    activeSpecs[j].SetNextSpecPtr((j < activeSpecs.Count - 1)
        ? newSpecOffset + specEntitySize
        : -1);

    _specSerializer.WriteToFile(activeSpecs[j], writer);
    writer.Flush();

    lastSpecOffset = (int)tempSpecStream.Position;
    _productHeader.SetUnclaimedPtr(lastSpecOffset);
}
tempSpecStream.Seek(unclaimedPtrPos, SeekOrigin.Begin);
writer.Write(lastSpecOffset);
writer.Flush();

tempSpecStream.Seek(0, SeekOrigin.End);
}
else
{
    sortedProducts[i].SetSpecPtr(-1);
}
}
else
{
    sortedProducts[i].SetSpecPtr(-1);
}
}
}
for (int i = 0; i < sortedProducts.Count; i++)
{
    sortedProducts[i].SetNextProductPtr((i < sortedProducts.Count - 1)
        ? sortedProducts[i].FileOffset + productEntitySize
        : -1);
}

tempProductStream.Seek(0, SeekOrigin.Begin);
using (var writer = new BinaryWriter(tempProductStream, Encoding.UTF8,
leaveOpen: true))
{
    writer.Write(Encoding.ASCII.GetBytes(_productHeader.Signature.PadRight(2).Substring(0,
2)));

    writer.Write(_productHeader.DataLength);
    writer.Write(_productHeader.FirstRecPtr);
    writer.Write(_productHeader.UnclaimedPtr);

    if (!string.IsNullOrEmpty(_productHeader.SpecFileName))
    {
        char[] specBuffer = new char[16];
        Array.Copy(_productHeader.SpecFileName.ToCharArray(), specBuffer,

```

```

        Math.Min(_productHeader.SpecFileName.Length, 16));
        writer.Write(specBuffer);
    }
}

for (int i = 0; i < sortedProducts.Count; i++)
{
    tempProductStream.Seek(sortedProducts[i].FileOffset, SeekOrigin.Begin);
    using (var writer = new BinaryWriter(tempProductStream, Encoding.UTF8,
leaveOpen: true))
    {
        _productSerializer.WriteToFile(sortedProducts[i], writer);
    }
}

tempProductStream.Flush();
tempSpecStream.Flush();
}
catch
{
    if (File.Exists(_productFsManager.GetTempFilePath()))
        File.Delete(_productFsManager.GetTempFilePath());
    if (File.Exists(_specFsManager.GetTempFilePath()))
        File.Delete(_specFsManager.GetTempFilePath());
    throw;
}
finally
{
    tempProductStream?.Dispose();
    tempSpecStream?.Dispose();
}

_productFsManager.ReplaceWithTempFile();
_specFsManager.ReplaceWithTempFile();

_productCache = sortedProducts.ToDictionary(p => p.Name, p => p);

var newSpecListCache = new Dictionary<int, List<Spec>>>();
for (int i = 0; i < sortedProducts.Count; i++)
{
    int oldProductOffset = oldProductOffsets[i];
    if (_specListCache.TryGetValue(oldProductOffset, out var specs))
    {
        var activeSpecs = specs.Where(s => !s.IsDeleted).ToList();
        if (activeSpecs.Count > 0)
        {
            newSpecListCache[sortedProducts[i].FileOffset] = activeSpecs;
        }
    }
}
specListCache = newSpecListCache;

```

```

        _specHeaderCache.Clear();
        foreach (var product in sortedProducts)
        {
            if (_specListCache.ContainsKey(product.FileOffset))
            {
                var specs = _specListCache[product.FileOffset];
                int firstSpecPtr = specs.Count > 0 ? specs[0].FileOffset : -1;
                int unclaimedPtr = specs.Count > 0 ? specs[specs.Count - 1].FileOffset +
specEntitySize : SpecHeader.GetHeaderSize();
                _specHeaderCache[product.FileOffset] = new SpecHeader(firstSpecPtr,
unclaimedPtr);
            }
        }
    }

    public SpecHeader GetSpecHeader(int productOffset)
    {
        if (_specHeaderCache.ContainsKey(productOffset))
            return _specHeaderCache[productOffset];
        return null;
    }

    public IEnumerable<Spec> GetSpecsForProduct(int productOffset)
    {
        if (_specListCache.ContainsKey(productOffset))
            return _specListCache[productOffset]
                .Where(s => !s.IsDeleted)
                .OrderBy(s => s.Name)
                .ToList();
        return Enumerable.Empty<Spec>();
    }

    public void EditProduct(string oldName, string newName)
    {
        if (!_productCache.TryGetValue(oldName, out var product))
            throw new InvalidOperationException("Component not found.");

        if (oldName != newName && _productCache.ContainsKey(newName))
            throw new InvalidOperationException("Component with this name already exists.");

        product.SetComponentName(newName);
        UpdateProduct(product);

        _productCache.Remove(oldName);
        _productCache[newName] = product;
        SortAlphabetically();
    }

    public void EditSpec(int productOffset, int specComponentPtr, ushort newMultiplicity)
    {

```

```

        if (!_specListCache.ContainsKey(productOffset))
            throw new InvalidOperationException("Product specifications not found.");

        var spec = _specListCache[productOffset]
            .FirstOrDefault(s => s.ComponentPtr == specComponentPtr && !s.IsDeleted);

        if (spec == null)
            throw new InvalidOperationException("Specification not found.");

        spec.SetMultiplicity(new Multiplicity);
        UpdateSpec(spec);
    }
}
}

```

ProductFSManager.cs

```

using ConsoleApp2.Entities;
using System;
using System.Collections.Generic;
using System.Text;
using System.IO;
using ConsoleApp2.Data.Abstractions;

namespace ConsoleApp2.Data
{
    public class ProductFSManager : BaseFSManager
    {
        public ProductFSManager(string filePath) : base(filePath)
        {
        }

        public void WriteHeader(ProductHeader header)
        {
            if (_fileStream == null)
                throw new InvalidOperationException("File not opened.");

            using (var writer = new BinaryWriter(_fileStream, Encoding.UTF8, leaveOpen: true))
            {
                _fileStream.Seek(0, SeekOrigin.Begin);
                writer.Write(Encoding.ASCII.GetBytes(header.Signature.PadRight(2).Substring(0,
2)));
                writer.Write(header.DataLength);
                writer.Write(header.FirstRecPtr);
                writer.Write(header.UnclaimedPtr);

                if (!string.IsNullOrEmpty(header.SpecFileName))
                {
                    char[] specBuffer = new char[16];
                    Array.Copy(header.SpecFileName.ToCharArray(), specBuffer,

```

```

        Math.Min(header.SpecFileName.Length, 16));
        writer.Write(specBuffer);
    }
}
_fileStream.Flush();
}

public ProductHeader ReadHeader()
{
    if (_fileStream == null)
        throw new InvalidOperationException("File not opened.");

    using (var reader = new BinaryReader(_fileStream, Encoding.UTF8, leaveOpen: true))
    {
        _fileStream.Seek(0, SeekOrigin.Begin);
        byte[] sig = reader.ReadBytes(2);
        string signature = Encoding.ASCII.GetString(sig);

        if (signature != "PS")
            throw new InvalidOperationException("Invalid file signature.");

        short dataLength = reader.ReadInt16();
        int firstRecPtr = reader.ReadInt32();
        int unclaimedPtr = reader.ReadInt32();
        char[] specName = reader.ReadChars(16);

        string specFileName = new string(specName).TrimEnd('\0');

        var header = new ProductHeader(signature, dataLength, firstRecPtr, unclaimedPtr,
specFileName);

        return header;
    }
}

public ProductHeader ReadProductHeader()
{
    return ReadHeader();
}
}
}

```

ProductSerializer.cs

```

using ConsoleApp2.Data.Abstractions;
using ConsoleApp2.Entities;
using System;
using System.IO;
using System.Text;

```

```

namespace ConsoleApp2.Data
{
    public class ProductSerializer : ISerializer<Product>
    {
        private short _dataLength;

        public ProductSerializer(short dataLength)
        {
            if (dataLength <= 0)
                throw new ArgumentException("Data length must be greater than zero.",
nameof(dataLength));
            _dataLength = dataLength;
        }

        public void WriteToFile(Product product, BinaryWriter writer)
        {
            if (product == null)
                throw new ArgumentNullException(nameof(product), "Product is null.");

            writer.Write(product.DelBit);
            writer.Write((byte)product.Type);
            writer.Write(product.SpecPtr);
            writer.Write(product.NextProductPtr);

            byte[] nameBytes = Encoding.UTF8.GetBytes(product.Name ?? string.Empty);
            byte[] buffer = new byte[_dataLength];
            Array.Copy(nameBytes, buffer, Math.Min(nameBytes.Length, _dataLength));
            writer.Write(buffer);
        }

        public Product ReadFromFile(BinaryReader reader)
        {
            if (reader == null)
                throw new ArgumentNullException(nameof(reader), "Reader is null.");

            byte delBit = reader.ReadByte();
            ComponentType type = (ComponentType)reader.ReadByte();
            int specPtr = reader.ReadInt32();
            int nextPtr = reader.ReadInt32();
            byte[] nameBytes = reader.ReadBytes(_dataLength);
            string name = Encoding.UTF8.GetString(nameBytes).TrimEnd('\0', ' ');

            Product product = new Product(name, type)
            {
                DelBit = delBit,
                SpecPtr = specPtr,
                NextProductPtr = nextPtr
            };

            return product;
        }
    }
}

```

```

        public int GetEntitySize() => FileStructure.DELBIT_SIZE + FileStructure.TYPE_SIZE +
        FileStructure.POINTER_SIZE + FileStructure.POINTER_SIZE + _dataLength;
    }
}

```

Repository.cs

```

using ConsoleApp2.Data.Abstractions;
using ConsoleApp2.Entities;
using System;
using System.Collections.Generic;
using System.Text;
using System.Linq;
using System.IO;

namespace ConsoleApp2.Data
{
    public class Repository : IRepository
    {
        private ProductFSManager _productFsManager;
        private SpecFSManager _specFsManager;
        private LLManager _listManager;
        private ProductHeader _productHeader;
        private ProductSerializer _productSerializer;
        private SpecSerializer _specSerializer;

        public Repository(ProductFSManager productFsManager, SpecFSManager
specFsManager,
            ProductSerializer productSerializer, SpecSerializer specSerializer)
        {
            _productFsManager = productFsManager ?? throw new
ArgumentNullException(nameof(productFsManager), "Product file manager is null.");
            _specFsManager = specFsManager ?? throw new
ArgumentNullException(nameof(specFsManager), "Spec file manager is null.");
            _productSerializer = productSerializer ?? throw new
ArgumentNullException(nameof(productSerializer), "Product serializer is null.");
            _specSerializer = specSerializer ?? throw new
ArgumentNullException(nameof(specSerializer), "Spec serializer is null.");
            _listManager = new LLManager(productFsManager, specFsManager,
productSerializer, specSerializer);
        }

        public void Create(string filePath, short dataLength, string specFileName)
        {
            if (!filePath.EndsWith(".prd"))
                filePath += ".prd";

            string dir = Path.GetDirectoryName(filePath);
            if (string.IsNullOrEmpty(dir))

```

```

        dir = ".";

        string specFilePath = Path.Combine(dir, specFileName);

        _productFsManager.CreateFile();
        _specFsManager.CreateFile();

        _productHeader = new ProductHeader(dataLength,
Path.GetFileName(specFileName));
        _productHeader.FirstRecPtr = -1;
        _productHeader.UnclaimedPtr = ProductHeader.GetHeaderSize();

        _listManager.Initialize(_productHeader);

        _productFsManager.WriteHeader(_productHeader);
    }

    public void Open(string filePath)
    {
        if (!filePath.EndsWith(".prd"))
            filePath += ".prd";

        if (_productFsManager.GetStream() == null)
        {
            _productFsManager.OpenFile();
        }

        _productHeader = _productFsManager.ReadHeader();

        string dir = Path.GetDirectoryName(filePath);
        if (string.IsNullOrEmpty(dir))
            dir = ".";

        string specFilePath = Path.Combine(dir, _productHeader.SpecFileName);

        if (_specFsManager.GetStream() == null)
        {
            _specFsManager.OpenFile();
        }

        _listManager.Initialize(_productHeader);
        _listManager.LoadFromFile();
    }

    public void Close()
    {
        _productFsManager?.Close();
        _specFsManager?.Close();
    }

    public void Save()

```

```

{
    if (_productHeader != null)
        _productFsManager.WriteHeader(_productHeader);
}

public void Add(Product product)
{
    if (product == null)
        throw new ArgumentNullException(nameof(product), "Product is null.");

    if (_listManager == null)
        throw new InvalidOperationException("Repository not initialized. Call Create or
Open first.");

    if (_productFsManager == null || _productFsManager.GetStream() == null)
        throw new InvalidOperationException("Product file not open.");

    if (_specFsManager == null || _specFsManager.GetStream() == null)
        throw new InvalidOperationException("Spec file not open.");

    if (_listManager.FindByName(product.Name) != null)
        throw new InvalidOperationException("Component already exists.");

    int headerSize = ProductHeader.GetHeaderSize();
    int offset = (int)_productFsManager.GetStream().Length;

    if (offset < headerSize)
        offset = headerSize;

    product.FileOffset = offset;

    int specHeaderOffset = (int)_specFsManager.GetStream().Length;
    if (specHeaderOffset == 0)
        specHeaderOffset = 0;

    product.SpecPtr = specHeaderOffset;

    if (_productHeader.FirstRecPtr == -1)
    {
        _productHeader.FirstRecPtr = offset;
    }
    else
    {
        var lastProduct = _listManager.GetLastProduct();
        if (lastProduct != null)
        {
            lastProduct.NextProductPtr = offset;
            _listManager.UpdateProduct(lastProduct);
        }
    }
}

```

```

        _listManager.AddProduct(product);

        _specFsManager.Seek(specHeaderOffset);
        var specHeader = new SpecHeader(-1, specHeaderOffset +
SpecHeader.GetHeaderSize());
        using (var writer = new BinaryWriter(_specFsManager.GetStream(), Encoding.UTF8,
leaveOpen: true))
        {
            writer.Write(specHeader.FirstRecPtr);
            writer.Write(specHeader.UnclaimedPtr);
        }
        _specFsManager.GetStream().Flush();

        _listManager.UpdateProduct(product);
        _productFsManager.WriteHeader(_productHeader);
    }

    public void Delete(string productName)
    {
        var product = _listManager.FindByName(productName);
        if (product == null)
            throw new InvalidOperationException("Component not found.");

        var allProducts = _listManager.GetAllProducts().ToList();
        foreach (var parentProduct in allProducts)
        {
            if (parentProduct.FileOffset == product.FileOffset)
                continue;

            var parentSpecs =
_listManager.GetSpecsForProduct(parentProduct.FileOffset).Where(s =>
!s.IsDeleted).ToList();
            foreach (var spec in parentSpecs)
            {
                if (spec.ComponentPtr == product.FileOffset)
                {
                    throw new InvalidOperationException($"Component is referenced by
{parentProduct.Name}.");
                }
            }
        }

        _listManager.Delete(product.FileOffset);
    }

    public void Restore(string productName)
    {
        _listManager.Restore(productName);
        _listManager.SortAlphabetically();
    }

```

```

public void RestoreAll()
{
    _listManager.RestoreAllProducts();
    _listManager.RestoreAllSpecs();
    _listManager.SortAlphabetically();
}

public void Truncate()
{
    _listManager.Truncate();
}

public Product Find(string productName)
{
    return _listManager.FindByName(productName);
}

public Product FindIncludeDeleted(string productName)
{
    return _listManager.FindByNameIncludeDeleted(productName);
}

public IEnumerable<Product> GetAll()
{
    return _listManager.GetAllProducts();
}

public void AddSpec(Product product, Spec spec)
{
    if (spec == null)
        throw new ArgumentNullException(nameof(spec));

    var specHeader = _listManager.GetSpecHeader(product.FileOffset);

    if (specHeader == null)
    {
        int specHeaderOffset;
        bool isNewSpecPtr = false;

        if (product.SpecPtr == -1)
        {
            specHeaderOffset = (int)_specFsManager.GetStream().Length;
            product.SpecPtr = specHeaderOffset;
            isNewSpecPtr = true;
        }
        else
        {
            specHeaderOffset = product.SpecPtr;
        }
    }
}

```

```

        specHeader = new SpecHeader(-1, specHeaderOffset +
SpecHeader.GetHeaderSize());
        _specFsManager.Seek(specHeaderOffset);
        using (var writer = new BinaryWriter(_specFsManager.GetStream(),
Encoding.UTF8, leaveOpen: true))
        {
            writer.Write(specHeader.FirstRecPtr);
            writer.Write(specHeader.UnclaimedPtr);
        }
        _specFsManager.GetStream().Flush();

        if (isNewSpecPtr)
        {
            _listManager.UpdateProduct(product);
        }
    }

    int specOffset = (int)_specFsManager.GetStream().Length;
    spec.FileOffset = specOffset;

    if (specHeader.FirstRecPtr != -1)
    {
        int currentOffset = specHeader.FirstRecPtr;
        Spec lastSpec = null;

        while (currentOffset != -1 && currentOffset != 1)
        {
            var s = _listManager.FindSpecByOffset(currentOffset);
            if (s == null)
                break;

            lastSpec = s;
            currentOffset = s.NextSpecPtr;
        }

        if (lastSpec != null)
        {
            lastSpec.NextSpecPtr = specOffset;
            _listManager.UpdateSpec(lastSpec);
        }
    }
    else
    {
        specHeader.FirstRecPtr = specOffset;
        _specFsManager.Seek(product.SpecPtr);
        using (var writer = new BinaryWriter(_specFsManager.GetStream(),
Encoding.UTF8, leaveOpen: true))
        {
            writer.Write(specHeader.FirstRecPtr);
            writer.Write(specHeader.UnclaimedPtr);
        }
    }

```

```

        _specFsManager.GetStream().Flush();
    }

    _listManager.AddSpec(product.FileOffset, spec);
}

public void DeleteSpec(Product product, string specName)
{
    var specComponent = _listManager.FindByName(specName);
    if (specComponent == null)
        throw new InvalidOperationException("Component not found.");

    var specHeader = _listManager.GetSpecHeader(product.FileOffset);
    if (specHeader == null)
        throw new InvalidOperationException("Specification not found.");

    int currentOffset = specHeader.FirstRecPtr;
    Spec specToDelete = null;
    Spec prevSpec = null;

    while (currentOffset != -1 && currentOffset != 1)
    {
        var spec = _listManager.FindSpecByOffset(currentOffset);
        if (spec == null)
            break;

        if (spec.ComponentPtr == specComponent.FileOffset)
        {
            specToDelete = spec;
            break;
        }

        prevSpec = spec;
        currentOffset = spec.NextSpecPtr;
    }

    if (specToDelete == null)
        throw new InvalidOperationException("Specification not found.");

    specToDelete.MarkAsDeleted();
    _listManager.UpdateSpec(specToDelete);

    if (prevSpec != null)
    {
        prevSpec.NextSpecPtr = specToDelete.NextSpecPtr;
        _listManager.UpdateSpec(prevSpec);
    }
    else if (specHeader.FirstRecPtr == specToDelete.FileOffset)
    {
        specHeader.FirstRecPtr = specToDelete.NextSpecPtr;
        _specFsManager.Seek(product.SpecPtr);
    }
}

```

```

        using (var writer = new BinaryWriter(_specFsManager.GetStream(),
Encoding.UTF8, leaveOpen: true))
        {
            writer.Write(specHeader.FirstRecPtr);
            writer.Write(specHeader.UnclaimedPtr);
        }
        _specFsManager.GetStream().Flush();
    }
}

public Spec FindSpecByOffset(int offset)
{
    return _listManager.FindSpecByOffset(offset);
}

public IEnumerable<Spec> GetAllSpecs()
{
    return Enumerable.Empty<Spec>();
}

public IEnumerable<Spec> GetSpecsForProduct(int productOffset)
{
    return _listManager.GetSpecsForProduct(productOffset);
}

public void EditProduct(string oldName, string newName)
{
    var product = _listManager.FindByName(oldName);
    if (product == null)
        throw new InvalidOperationException("Component not found.");

    var existingProduct = _listManager.FindByName(newName);
    if (existingProduct != null && existingProduct.FileOffset != product.FileOffset)
        throw new InvalidOperationException("Component with this name already exists.");

    _listManager.EditProduct(oldName, newName);
}

public void EditSpec(Product product, string specName, ushort newMultiplicity)
{
    var specComponent = _listManager.FindByName(specName);
    if (specComponent == null)
        throw new InvalidOperationException("Specification component not found.");

    _listManager.EditSpec(product.FileOffset, specComponent.FileOffset,
newMultiplicity);
}

public void ChangeSpecFileName(string newSpecFileName)
{
    _productHeader.SetSpecFileName(newSpecFileName);
}

```

```
        _productFsManager.WriteHeader(_productHeader);
    }
}
}
```

SpecFSManager.cs

```
using System;
using System.Collections.Generic;
using System.Text;
using System.IO;
using ConsoleApp2.Data.Abstractions;
using ConsoleApp2.Entities;

namespace ConsoleApp2.Data
{
    public class SpecFSManager : BaseFSManager
    {
        public SpecFSManager(string filePath) : base(filePath)
        {
        }

        public void WriteHeader(SpecHeader header)
        {
            if (_fileStream == null)
                throw new InvalidOperationException("File not opened.");

            using (var writer = new BinaryWriter(_fileStream, Encoding.UTF8, leaveOpen: true))
            {
                _fileStream.Seek(0, SeekOrigin.Begin);
                writer.Write(header.FirstRecPtr);
                writer.Write(header.UnclaimedPtr);
            }
            _fileStream.Flush();
        }

        public SpecHeader ReadHeader()
        {
            if (_fileStream == null)
                throw new InvalidOperationException("File not opened.");

            using (var reader = new BinaryReader(_fileStream, Encoding.UTF8, leaveOpen: true))
            {
                _fileStream.Seek(0, SeekOrigin.Begin);
                int firstRecPtr = reader.ReadInt32();
                int unclaimedPtr = reader.ReadInt32();
                var header = new SpecHeader(firstRecPtr, unclaimedPtr);
                return header;
            }
        }
    }
}
```

```
}  
}
```

SpecSerializer.cs

```
using ConsoleApp2.Data.Abstractions;  
using ConsoleApp2.Entities;  
using System;  
using System.IO;  
  
namespace ConsoleApp2.Data  
{  
    public class SpecSerializer : ISerializer<Spec>  
    {  
        public void WriteToFile(Spec entity, BinaryWriter writer)  
        {  
            if (entity == null)  
                throw new ArgumentNullException(nameof(entity), "Spec is null.");  
  
            writer.Write(entity.DelBit);  
            writer.Write(entity.ComponentPtr);  
            writer.Write(entity.Multiplicity);  
            writer.Write(entity.NextSpecPtr);  
        }  
  
        public Spec ReadFromFile(BinaryReader reader)  
        {  
            if (reader == null)  
                throw new ArgumentNullException(nameof(reader), "Reader is null.");  
  
            byte delBit = reader.ReadByte();  
            int componentPtr = reader.ReadInt32();  
            ushort multiplicity = reader.ReadUInt16();  
            int nextPtr = reader.ReadInt32();  
  
            Spec spec = new Spec(componentPtr, multiplicity)  
            {  
                DelBit = delBit,  
                NextSpecPtr = nextPtr  
            };  
  
            return spec;  
        }  
  
        public int GetEntitySize() => FileStructure.DELBIT_SIZE + FileStructure.POINTER_SIZE  
        + FileStructure.MULTIPLICITY_SIZE + FileStructure.POINTER_SIZE;  
    }  
}
```

FileHeader.cs

```
using System;

namespace ConsoleApp2.Entities
{
    public class FileHeader
    {
        public int FirstRecPtr { get; set; } = -1;
        public int UnclaimedPtr { get; set; } = -1;
    }
}
```

Product.cs

```
using System;
using System.Collections.Generic;
using System.Text;

namespace ConsoleApp2.Entities
{
    public enum ComponentType
    {
        Product,
        Node,
        Detail
    }

    public interface IDeletable
    {
        byte DelBit { get; }
        void MarkAsDeleted();
        void Restore();
        bool IsDeleted { get; }
    }

    public interface IEntity : IDeletable
    {
        string Name { get; }
        int FileOffset { get; set; }
    }

    public class Product : IEntity
    {
        private byte _delBit;
        private int _specPtr;
        private int _nextProductPtr;
        private int _fileOffset;

        public byte DelBit
```

```

{
    get { return _delBit; }
    set { _delBit = value; }
}

public int SpecPtr
{
    get { return _specPtr; }
    set { _specPtr = value; }
}

public int NextProductPtr
{
    get { return _nextProductPtr; }
    set { _nextProductPtr = value; }
}

public string Name { get; private set; }

public ComponentType Type { get; private set; }

public int FileOffset
{
    get { return _fileOffset; }
    set { _fileOffset = value; }
}

public bool IsDeleted => DelBit == 1;

public Product(string name, ComponentType type)
{
    if (string.IsNullOrEmpty(name))
        throw new ArgumentException("Component name cannot be null or empty.");

    _delBit = 0;
    _specPtr = -1;
    _nextProductPtr = -1;
    Name = name;
    Type = type;
    _fileOffset = -1;
}

public void SetSpecPtr(int specPtr)
{
    SpecPtr = specPtr;
}

public void SetNextProductPtr(int nextProductPtr)
{
    NextProductPtr = nextProductPtr;
}

```

```

    public void SetFileOffset(int fileOffset)
    {
        if (fileOffset < -1)
            throw new ArgumentException("File offset must be -1 or greater.");
        FileOffset = fileOffset;
    }

    public void SetComponentName(string newName)
    {
        if (string.IsNullOrEmpty(newName))
            throw new ArgumentException("Component name cannot be null or empty.");
        Name = newName;
    }

    public void MarkAsDeleted() => _delBit = 1;
    public void Restore() => _delBit = 0;
}

```

ProductHeader.cs

```

using System;
using System.Collections.Generic;
using System.Text;
using ConsoleApp2.Data.Abstractions;

namespace ConsoleApp2.Entities
{
    public class ProductHeader
    {
        private string _signature;
        private short _dataLength;
        private int _firstRecPtr;
        private int _unclaimedPtr;
        private string _specFileName;

        public string Signature
        {
            get { return _signature; }
            private set { _signature = value; }
        }

        public short DataLength
        {
            get { return _dataLength; }
            private set
            {
                if ((value <= 0) || (value > 32000))

```

```

        throw new ArgumentOutOfRangeException(nameof(value), "Data length must be
between 1 and 32000.");
        _dataLength = value;
    }
}

public int FirstRecPtr
{
    get { return _firstRecPtr; }
    set { _firstRecPtr = value; }
}

public int UnclaimedPtr
{
    get { return _unclaimedPtr; }
    set { _unclaimedPtr = value; }
}

public string SpecFileName
{
    get { return _specFileName; }
    private set { _specFileName = value; }
}

public ProductHeader(short dataLength, string specFileName)
{
    SetDataLength(dataLength);
    Signature = FileStructure.DEFAULT_SIGNATURE;
    _firstRecPtr = -1;
    _unclaimedPtr = -1;
    SetSpecFileName(specFileName);
}

public ProductHeader(string signature, short dataLength, int firstRecPtr, int
unclaimedPtr, string specFileName)
{
    if (string.IsNullOrEmpty(signature))
        throw new ArgumentException("Signature cannot be null or empty.");

    Signature = signature;
    SetDataLength(dataLength);
    _firstRecPtr = firstRecPtr;
    _unclaimedPtr = unclaimedPtr;
    SetSpecFileName(specFileName);
}

public void SetDataLength(short dataLength)
{
    DataLength = dataLength;
}

```

```

    public void SetFirstRecPtr(int firstRecPtr)
    {
        FirstRecPtr = firstRecPtr;
    }

    public void SetUnclaimedPtr(int unclaimedPtr)
    {
        UnclaimedPtr = unclaimedPtr;
    }

    public void SetSpecFileName(string specFileName)
    {
        if (string.IsNullOrEmpty(specFileName))
            throw new ArgumentException("Specification file name cannot be null or empty.");
        SpecFileName = specFileName;
    }

    public static int GetHeaderSize()
    {
        return FileStructure.SIGNATURE_SIZE + FileStructure.DATA_LENGTH_SIZE +
            FileStructure.POINTER_SIZE + FileStructure.POINTER_SIZE +
            FileStructure.SPEC_FILENAME_SIZE;
    }
}

```

Spec.cs

```

using System;
using System.Collections.Generic;
using System.Text;

namespace ConsoleApp2.Entities
{
    public class Spec : IEntity
    {
        private byte _delBit;
        private int _componentPtr;
        private ushort _multiplicity;
        private int _nextSpecPtr;
        private int _fileOffset;
        private string _name;

        public byte DelBit
        {
            get { return _delBit; }
            set { _delBit = value; }
        }

        public int ComponentPtr

```

```

{
    get { return _componentPtr; }
    private set { _componentPtr = value; }
}

public ushort Multiplicity
{
    get { return _multiplicity; }
    private set { _multiplicity = value; }
}

public int NextSpecPtr
{
    get { return _nextSpecPtr; }
    set { _nextSpecPtr = value; }
}

public int FileOffset
{
    get { return _fileOffset; }
    set { _fileOffset = value; }
}

public string Name
{
    get { return _name; }
    private set { _name = value; }
}

public bool IsDeleted => DelBit == 1;

public Spec(int componentPtr, ushort multiplicity = 1)
{
    _delBit = 0;
    SetComponentPtr(componentPtr);
    SetMultiplicity(multiplicity);
    _nextSpecPtr = -1;
    _fileOffset = -1;
    _name = null;
}

public void SetComponentPtr(int componentPtr)
{
    if (componentPtr < 0)
        throw new ArgumentException("Component pointer must be non-negative.");
    ComponentPtr = componentPtr;
}

public void SetMultiplicity(ushort multiplicity)
{
    if (multiplicity <= 0)

```

```

        throw new ArgumentException("Multiplicity must be greater than zero.");
        Multiplicity = multiplicity;
    }

    public void SetNextSpecPtr(int nextSpecPtr)
    {
        NextSpecPtr = nextSpecPtr;
    }

    public void SetFileOffset(int fileOffset)
    {
        if (fileOffset < -1)
            throw new ArgumentException("File offset must be -1 or greater.");
        FileOffset = fileOffset;
    }

    public void SetName(string name)
    {
        Name = name;
    }

    public void MarkAsDeleted() => _delBit = 1;
    public void Restore() => _delBit = 0;
    }
}

```

SpecHeader.cs

```

using System;
using System.Collections.Generic;
using System.Text;
using ConsoleApp2.Data.Abstractions;

namespace ConsoleApp2.Entities
{
    public class SpecHeader
    {
        private int _firstRecPtr;
        private int _unclaimedPtr;

        public int FirstRecPtr
        {
            get { return _firstRecPtr; }
            set { _firstRecPtr = value; }
        }

        public int UnclaimedPtr
        {
            get { return _unclaimedPtr; }
            set { _unclaimedPtr = value; }
        }
    }
}

```

```

    }

    public SpecHeader(int firstRecPtr, int unclaimedPtr)
    {
        SetFirstRecPtr(firstRecPtr);
        SetUnclaimedPtr(unclaimedPtr);
    }

    public void SetFirstRecPtr(int firstRecPtr)
    {
        FirstRecPtr = firstRecPtr;
    }

    public void SetUnclaimedPtr(int unclaimedPtr)
    {
        if (unclaimedPtr < 0)
            throw new ArgumentException("Unclaimed pointer must be non-negative.");
        UnclaimedPtr = unclaimedPtr;
    }

    public static int GetHeaderSize()
    {
        return FileStructure.POINTER_SIZE + FileStructure.POINTER_SIZE;
    }
}

```

ConsoleIPSMenu.cs

```

using System;
using System.Collections.Generic;
using System.Text;
using ConsoleApp2.Data.Abstractions;

namespace ConsoleApp2.Entities
{
    public class SpecHeader
    {
        private int _firstRecPtr;
        private int _unclaimedPtr;

        public int FirstRecPtr
        {
            get { return _firstRecPtr; }
            set { _firstRecPtr = value; }
        }

        public int UnclaimedPtr
        {
            get { return _unclaimedPtr; }

```

```

        set { _unclaimedPtr = value; }
    }

    public SpecHeader(int firstRecPtr, int unclaimedPtr)
    {
        SetFirstRecPtr(firstRecPtr);
        SetUnclaimedPtr(unclaimedPtr);
    }

    public void SetFirstRecPtr(int firstRecPtr)
    {
        FirstRecPtr = firstRecPtr;
    }

    public void SetUnclaimedPtr(int unclaimedPtr)
    {
        if (unclaimedPtr < 0)
            throw new ArgumentException("Unclaimed pointer must be non-negative.");
        UnclaimedPtr = unclaimedPtr;
    }

    public static int GetHeaderSize()
    {
        return FileStructure.POINTER_SIZE + FileStructure.POINTER_SIZE;
    }
}

```

IMenu.cs

```

using System;
using System.Collections.Generic;
using System.Text;

namespace ConsoleApp2.Menu
{
    internal interface IMenu
    {
        void Show();
    }

    internal interface IMenuItem : IInvokeWithStringInput
    {
        public string Title { get; }
    }

    internal interface IInvokeWithStringInput
    {
        public bool Invoke(string input);
    }
}

```

```
internal interface IMenuItemStringable : IMenuItem, IInvokeWithStringInput { }  
}
```

FileService.cs

```
using ConsoleApp2.Data;  
using ConsoleApp2.Data.Abstractions;  
using ConsoleApp2.Entities;  
using System;  
using System.Collections.Generic;  
using System.Linq;  
  
namespace ConsoleApp2.MenuService  
{  
    public class FileService : IFileService  
    {  
        protected Repository _productRepo;  
        private bool _filesOpen;  
        public event EventHandler ProductsChanged;  
        public event EventHandler<string> ErrorOccurred;  
        public bool IsFilesOpen => _filesOpen;  
  
        public FileService()  
        {  
            _productRepo = null!;  
            _filesOpen = false;  
        }  
  
        public virtual void Create(string directoryPath, string productFileName, string  
specFileName, short dataLength)  
        {  
            try  
            {  
                if (string.IsNullOrEmpty(directoryPath))  
                    throw new ArgumentException("Directory path cannot be null or empty.");  
                if (string.IsNullOrEmpty(productFileName))  
                    throw new ArgumentException("Product file name cannot be null or empty.");  
                if (string.IsNullOrEmpty(specFileName))  
                    throw new ArgumentException("Spec file name cannot be null or empty.");  
                if (dataLength <= 0 || dataLength > 32000)  
                    throw new ArgumentException("Data length must be between 1 and 32000.");  
  
                CloseWithoutVirtual();  
                string fullProductPath = Path.Combine(directoryPath, productFileName + ".prd");  
                string fullSpecPath = Path.Combine(directoryPath, specFileName + ".prs");  
  
                if (File.Exists(fullProductPath) || File.Exists(fullSpecPath))  
                {  

```

```

        Console.WriteLine("File exists. Overwrite. (y/n): ");
        if (Console.ReadLine()?.ToLower() != "y")
            return;
    }

    Directory.CreateDirectory(directoryPath);

    var productFs = new ProductFSManager(fullProductPath);
    var productSerializer = new ProductSerializer(dataLength);
    var specFs = new SpecFSManager(fullSpecPath);
    var specSerializer = new SpecSerializer();

    _productRepo = new Repository(productFs, specFs, productSerializer,
specSerializer);
    _productRepo.Create(fullProductPath, dataLength, specFileName + ".prs");

    _filesOpen = true;
    OnProductsChanged();
}
catch (OutOfMemoryException)
{
    OnError("No available memory to create files.");
}
catch (IOException ex)
{
    if (IsDiskFullError(ex))
        OnError("No available disk space to create files.");
    else
        OnError($"File operation error: {ex.Message}");
}
catch (Exception ex)
{
    OnError(ex.Message);
}
}

public virtual void Open(string fullProductPath)
{
    try
    {
        if (string.IsNullOrEmpty(fullProductPath))
            throw new ArgumentException("File path cannot be null or empty.");
        if (!fullProductPath.EndsWith(".prd", StringComparison.OrdinalIgnoreCase))
            throw new ArgumentException("Product file must have .prd extension.");
        if (!File.Exists(fullProductPath))
            throw new FileNotFoundException($"Product file not found: {fullProductPath}");

        CloseWithoutVirtual();

        string directory = Path.GetDirectoryName(fullProductPath);
        if (string.IsNullOrEmpty(directory))

```

```

        directory = ".";

        var productFs = new ProductFSManager(fullProductPath);
        productFs.OpenFile();
        var productHeader = productFs.ReadHeader();

        var productSerializer = new ProductSerializer(productHeader.DataLength);
        string specFileName = productHeader.SpecFileName;
        string specFilePath = Path.Combine(directory, specFileName);

        if (!File.Exists(specFilePath))
            throw new FileNotFoundException($"Spec file not found: {specFilePath}");

        var specFs = new SpecFSManager(specFilePath);
        var specSerializer = new SpecSerializer();

        _productRepo = new Repository(productFs, specFs, productSerializer,
specSerializer);
        _productRepo.Open(fullProductPath);

        _filesOpen = true;
        OnProductsChanged();
    }
    catch (OutOfMemoryException)
    {
        OnError("No available memory to open files.");
    }
    catch (IOException ex)
    {
        OnError($"File operation error: {ex.Message}");
    }
    catch (Exception ex)
    {
        OnError(ex.Message);
    }
}

public virtual void Close()
{
    try
    {
        if (_filesOpen && _productRepo != null)
        {
            _productRepo.Close();
            _filesOpen = false;
            _productRepo = null;
        }
    }
    catch (Exception ex)
    {
        OnError(ex.Message);
    }
}

```

```

    }
}

private void CloseWithoutVirtual()
{
    try
    {
        if (_filesOpen && _productRepo != null)
        {
            _productRepo.Close();
            _filesOpen = false;
            _productRepo = null;
        }
    }
    catch (Exception ex)
    {
        OnError(ex.Message);
    }
}

public virtual void Input(string componentName, ComponentType type)
{
    try
    {
        if (!ValidateFilesOpen())
            return;

        if (string.IsNullOrEmpty(componentName))
            throw new ArgumentException("Specify component name.");

        var product = new Product(componentName, type);
        _productRepo.Add(product);
        OnProductsChanged();
    }
    catch (OutOfMemoryException)
    {
        OnError("No available memory to add component.");
    }
    catch (IOException ex)
    {
        if (IsDiskFullError(ex))
            OnError("No available disk space to add component.");
        else
            OnError($"File operation error: {ex.Message}");
    }
    catch (Exception ex)
    {
        OnError(ex.Message);
    }
}

```

```

public virtual void Input(string componentName, string specificationName)
{
    Input(componentName, specificationName, 1);
}

public virtual void Input(string componentName, string specificationName, ushort
multiplicity)
{
    try
    {
        if (!ValidateFilesOpen())
            return;

        var component = _productRepo.Find(componentName);
        if (component == null)
            throw new InvalidOperationException("Component not found.");

        if (component.Type == ComponentType.Detail)
            throw new InvalidOperationException("Detail cannot contain components.");

        var specComponent = _productRepo.Find(specificationName);
        if (specComponent == null)
            throw new InvalidOperationException("Component not found.");

        if (multiplicity <= 0)
            throw new ArgumentException("Multiplicity must be greater than zero.");

        if (multiplicity > 32767)
            throw new ArgumentException("Multiplicity must not exceed 32767.");

        var spec = new Spec(specComponent.FileOffset, multiplicity);
        _productRepo.AddSpec(component, spec);
        OnProductsChanged();
    }
    catch (OutOfMemoryException)
    {
        OnError("No available memory to add specification.");
    }
    catch (IOException ex)
    {
        if (IsDiskFullError(ex))
            OnError("No available disk space to add specification.");
        else
            OnError($"File operation error: {ex.Message}");
    }
    catch (Exception ex)
    {
        OnError(ex.Message);
    }
}

```

```

public virtual void Delete(string componentName)
{
    try
    {
        if (!ValidateFilesOpen())
            return;

        var product = _productRepo.Find(componentName);
        if (product == null)
            throw new InvalidOperationException("Component not found.");

        _productRepo.Delete(componentName);
        OnProductsChanged();
    }
    catch (OutOfMemoryException)
    {
        OnError("No available memory to delete component.");
    }
    catch (IOException ex)
    {
        if (IsDiskFullError(ex))
            OnError("No available disk space to delete component.");
        else
            OnError($"File operation error: {ex.Message}");
    }
    catch (Exception ex)
    {
        OnError(ex.Message);
    }
}

public virtual void Delete(string componentName, string specificationName)
{
    try
    {
        if (!ValidateFilesOpen())
            return;

        var component = _productRepo.Find(componentName);
        if (component == null)
            throw new InvalidOperationException("Component not found.");

        if (component.Type == ComponentType.Detail)
            throw new InvalidOperationException("Detail cannot contain components.");

        var specComponent = _productRepo.Find(specificationName);
        if (specComponent == null)
            throw new InvalidOperationException("Component not found.");

        _productRepo.DeleteSpec(component, specificationName);
        OnProductsChanged();
    }
}

```

```

    }
    catch (OutOfMemoryException)
    {
        OnError("No available memory to delete specification.");
    }
    catch (IOException ex)
    {
        if (IsDiskFullError(ex))
            OnError("No available disk space to delete specification.");
        else
            OnError($"File operation error: {ex.Message}");
    }
    catch (Exception ex)
    {
        OnError(ex.Message);
    }
}

public virtual void Edit(string productName, string newProductName)
{
    try
    {
        if (!ValidateFilesOpen())
            return;

        if (string.IsNullOrEmpty(newProductName))
            throw new ArgumentException("New product name cannot be null or empty.");

        var product = _productRepo.Find(productName);
        if (product == null)
            throw new InvalidOperationException("Component not found.");

        var existingProduct = _productRepo.Find(newProductName);
        if (existingProduct != null && existingProduct.FileOffset != product.FileOffset)
            throw new InvalidOperationException("Component with this name already
exists.");

        _productRepo.EditProduct(productName, newProductName);
        OnProductsChanged();
    }
    catch (OutOfMemoryException)
    {
        OnError("No available memory to edit component.");
    }
    catch (IOException ex)
    {
        if (IsDiskFullError(ex))
            OnError("No available disk space to edit component.");
        else
            OnError($"File operation error: {ex.Message}");
    }
}

```

```

        catch (Exception ex)
        {
            OnError(ex.Message);
        }
    }

    public virtual void EditSpec(string productName, string specName, ushort
newMultiplicity)
    {
        try
        {
            if (!ValidateFilesOpen())
                return;

            var component = _productRepo.Find(productName);
            if (component == null)
                throw new InvalidOperationException("Component not found.");

            var specComponent = _productRepo.Find(specName);
            if (specComponent == null)
                throw new InvalidOperationException("Specification component not found.");

            if (newMultiplicity <= 0)
                throw new ArgumentException("Multiplicity must be greater than zero.");

            if (newMultiplicity > 65535)
                throw new ArgumentException("Multiplicity must not exceed 65535.");

            _productRepo.EditSpec(component, specName, newMultiplicity);
            OnProductsChanged();
        }
        catch (OutOfMemoryException)
        {
            OnError("No available memory to edit specification.");
        }
        catch (IOException ex)
        {
            if (IsDiskFullError(ex))
                OnError("No available disk space to edit specification.");
            else
                OnError($"File operation error: {ex.Message}");
        }
        catch (Exception ex)
        {
            OnError(ex.Message);
        }
    }

    public virtual void Restore(string componentName)
    {
        try

```

```

    {
        if (!ValidateFilesOpen())
            return;

        var product = _productRepo.FindIncludeDeleted(componentName);
        if (product == null)
            throw new InvalidOperationException("Component not found.");

        _productRepo.Restore(componentName);
        OnProductsChanged();
    }
    catch (OutOfMemoryException)
    {
        OnError("No available memory to restore component.");
    }
    catch (IOException ex)
    {
        if (IsDiskFullError(ex))
            OnError("No available disk space to restore component.");
        else
            OnError($"File operation error: {ex.Message}");
    }
    catch (Exception ex)
    {
        OnError(ex.Message);
    }
}

public virtual void Restore()
{
    try
    {
        if (!ValidateFilesOpen())
            return;

        _productRepo.RestoreAll();
        OnProductsChanged();
    }
    catch (OutOfMemoryException)
    {
        OnError("No available memory to restore all components.");
    }
    catch (IOException ex)
    {
        if (IsDiskFullError(ex))
            OnError("No available disk space to restore all components.");
        else
            OnError($"File operation error: {ex.Message}");
    }
    catch (Exception ex)
    {

```

```

        OnError(ex.Message);
    }
}

public virtual void Truncate()
{
    try
    {
        if (!ValidateFilesOpen())
            return;

        _productRepo.Truncate();
        OnProductsChanged();
    }
    catch (OutOfMemoryException)
    {
        OnError("No available memory to truncate files.");
    }
    catch (IOException ex)
    {
        if (IsDiskFullError(ex))
            OnError("No available disk space to truncate files.");
        else
            OnError($"File operation error: {ex.Message}");
    }
    catch (Exception ex)
    {
        OnError(ex.Message);
    }
}

public virtual void Print(string componentName)
{
    try
    {
        if (!ValidateFilesOpen())
            return;

        var component = _productRepo.Find(componentName);
        if (component == null)
            throw new InvalidOperationException("Component not found.");

        if (component.Type == ComponentType.Detail)
            throw new InvalidOperationException("Detail cannot contain components.");

        PrintComponentTree(component, "");
    }
    catch (Exception ex)
    {
        OnError(ex.Message);
    }
}

```

```

    }

    public void Print()
    {
        try
        {
            if (!ValidateFilesOpen())
                return;

            Console.WriteLine("{0,-20} {1}", "Name", "Type");
            foreach (var product in _productRepo.GetAll().OrderBy(p => p.Name))
            {
                Console.WriteLine("{0,-20} {1}", product.Name, product.Type);
            }
        }
        catch (Exception ex)
        {
            OnError(ex.Message);
        }
    }

    public void Help()
    {
        PrintHelp(null);
    }

    public void Help(string fileName)
    {
        PrintHelp(fileName);
    }

    public virtual void ChangeSpecFileName(string newSpecFileName)
    {
        try
        {
            if (!ValidateFilesOpen())
                return;

            if (string.IsNullOrEmpty(newSpecFileName))
                throw new ArgumentException("Specification file name cannot be null or empty.");

            _productRepo.ChangeSpecFileName(newSpecFileName);
            OnProductsChanged();
        }
        catch (Exception ex)
        {
            OnError(ex.Message);
        }
    }
}

```

```

public IEnumerable<Product> GetAllProducts()
{
    try
    {
        if (!_filesOpen)
            throw new InvalidOperationException("Files not open.");

        return _productRepo.GetAll();
    }
    catch (Exception ex)
    {
        OnError(ex.Message);
        return Enumerable.Empty<Product>();
    }
}

public Product GetProduct(string productName)
{
    try
    {
        if (!_filesOpen)
            throw new InvalidOperationException("Files not open.");

        return _productRepo.Find(productName);
    }
    catch (Exception ex)
    {
        OnError(ex.Message);
        return null;
    }
}

public IEnumerable<Spec> GetProductSpecifications(string productName)
{
    try
    {
        if (!_filesOpen)
            throw new InvalidOperationException("Files not open.");

        var product = _productRepo.Find(productName);
        if (product == null)
            throw new InvalidOperationException("Product not found.");

        return _productRepo.GetSpecsForProduct(product.FileOffset)
            .Where(s => !s.IsDeleted);
    }
    catch (Exception ex)
    {
        OnError(ex.Message);
        return Enumerable.Empty<Spec>();
    }
}

```

```

    }

    private void PrintHelp(string fileName)
    {
        var helpText = new[]
        {
            "Create <filename> <length> [spec_file] - Create files",
            "Open <filename> - Open files",
            "Input <name> <type> - Add component (Product, Node, Detail)",
            "Input <component(parent)> <component(child)> [multiplicity] - Add to parent's
specification",
            "Delete <name> - Delete component",
            "Delete <component(parent)> <component(child)> - Delete from parent's
specification",
            "Restore <name> - Restore component",
            "Restore (*) - Restore all",
            "Truncate - Physical deletion",
            "Print <name> - Show specification",
            "Print (*) - Show all components",
            "Help [file] - Help",
            "Exit - Exit"
        };

        if (string.IsNullOrEmpty(fileName))
        {
            foreach (var line in helpText)
                Console.WriteLine(line);
        }
        else
        {
            try
            {
                File.WriteAllLines(fileName, helpText);
            }
            catch (Exception ex)
            {
                OnError(ex.Message);
            }
        }
    }

    private bool ValidateFilesOpen()
    {
        if (!_filesOpen)
        {
            OnError("Files not open.");
            return false;
        }
        return true;
    }

```

```

private void PrintComponentTree(Product component, string prefix, bool printName =
true)
{
    if (printName)
        Console.WriteLine(prefix + component.Name);

    if (component.Type == ComponentType.Detail || component.SpecPtr == -1)
        return;

    var allProducts = _productRepo.GetAll().ToList();
    var allSpecs = _productRepo.GetSpecsForProduct(component.FileOffset).ToList();
    var specList = allSpecs.Where(s => !s.IsDeleted).ToList();

    for (int i = 0; i < specList.Count; i++)
    {
        var spec = specList[i];
        var child = allProducts.FirstOrDefault(p => p.FileOffset == spec.ComponentPtr);

        if (child != null)
        {
            string multiplicity = spec.Multiplicity > 1 ? " (x" + spec.Multiplicity + ")" : "";
            string connector = i == specList.Count - 1 ? "└─" : "├─";
            Console.WriteLine(prefix + connector + " " + child.Name + multiplicity);

            if (child.Type != ComponentType.Detail)
            {
                string newPrefix = prefix + (i == specList.Count - 1 ? "  " : " | ");
                PrintComponentTree(child, newPrefix, false);
            }
        }
    }
}

private void OnProductsChanged()
{
    ProductsChanged?.Invoke(this, EventArgs.Empty);
}

private bool IsDiskFullError(IOException ex)
{
    return ex.HResult == unchecked((int)0x80070070);
}

private void OnError(string message)
{
    ErrorOccurred?.Invoke(this, message);
}
}

```

IFileService.cs

```
using ConsoleApp2.Entities;
using System;
using System.Collections.Generic;

namespace ConsoleApp2.MenuService
{
    public interface IFileService
    {
        void Create(string directoryPath, string productFileName, string specFileName, short dataLength);
        void Open(string fullProductPath);
        void Close();
        void Input(string componentName, ComponentType type);
        void Input(string componentName, string specificationName);
        void Input(string componentName, string specificationName, ushort multiplicity);
        void Delete(string componentName);
        void Delete(string componentName, string specificationName);
        void Edit(string productName, string newProductName);
        void EditSpec(string productName, string specName, ushort newMultiplicity);
        void Restore(string componentName);
        void Restore();
        void Truncate();
        void Print(string componentName);
        void Print();
        void Help();
        void Help(string fileName);
        void ChangeSpecFileName(string newSpecFileName);

        IEnumerable<Product> GetAllProducts();
        Product GetProduct(string productName);
        IEnumerable<Spec> GetProductSpecifications(string productName);
        bool IsFilesOpen { get; }

        event EventHandler ProductsChanged;
        event EventHandler<string> ErrorOccurred;
    }
}
```

TempSaver.cs

```
using System;
using System.Collections.Generic;
using System.Text;
using System.IO;
using System.Globalization;

namespace ConsoleApp2.MenuService
{
```

```

public class TempSaver
{
    private string _path;

    public string? TempPath { get; private set; }

    public string? SpecTempPath { get; private set; }

    public TempSaver(string pathOriginal)
    {
        if (!File.Exists(pathOriginal))
        {
            throw new FileNotFoundException("File is not found.", pathOriginal);
        }

        _path = pathOriginal;
    }

    public void SaveToOrigin()
    {
        if (TempPath == null || TempPath == string.Empty
            || SpecTempPath == null || SpecTempPath == string.Empty)
        {
            throw new InvalidOperationException("Temp file does not exist.");
        }

        File.Copy(TempPath, _path, true);
        File.Copy(SpecTempPath, Path.Combine(Path.GetDirectoryName(_path) ??
Path.GetTempPath(),
        $"{Path.GetFileNameWithoutExtension(_path)}.prs"), true);
    }

    public void CreateTempFile()
    {
        TempPath = Path.Combine(Path.GetDirectoryName(_path) ?? Path.GetTempPath(),
        $"{Path.GetFileNameWithoutExtension(_path)}.temp.prd");
        File.Create(TempPath).Close();
        File.Copy(_path, TempPath, true);

        SpecTempPath = Path.Combine(Path.GetDirectoryName(_path) ??
Path.GetTempPath(),
        $"{Path.GetFileNameWithoutExtension(_path)}.temp.prs");
        File.Create(SpecTempPath).Close();
        File.Copy(Path.Combine(Path.GetDirectoryName(_path) ?? Path.GetTempPath(),
        $"{Path.GetFileNameWithoutExtension(_path)}.prs"), SpecTempPath, true);
    }

    public void DeleteTempFile()
    {
        if (File.Exists(TempPath))
        {

```

```

        File.Delete(TempPath);
    }

    if (File.Exists(SpecTempPath))
    {
        File.Delete(SpecTempPath);
    }

    TempPath = null;
    SpecTempPath = null;
}

~TempSaver()
{
    if (TempPath != null)
    {
        DeleteTempFile();
    }
}
}
}

```

Program.cs

```

using ConsoleApp2.MenuService;
using ConsoleApp2.Menu;
using System;
using System.Collections.Generic;

namespace ConsoleApp2
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;

            IFileService fileService = new FileService();

            IMenu menu = new ConsolPSMenu(fileService, new List<IMenuItemStringable>
            {
                new CreateMenuItem(fileService),
                new OpenMenuItem(fileService),
                new InputMenuItem(fileService),
                new DeleteMenuItem(fileService),
                new RestoreMenuItem(fileService),
                new TruncateMenuItem(fileService),
                new PrintMenuItem(fileService),
                new HelpMenuItem(fileService),
                new ExitMenuItem(fileService)
            }

```

```

    });

    menu.Show();
}
}
}

```

RelayCommand.cs

```

using System;
using System.Collections.Generic;
using System.Text;
using System.Windows.Input;

namespace PSListMaker.Commands
{
    public class RelayCommand : ICommand
    {
        private readonly Action _execute;
        private readonly Func<bool> _canExecute;

        public RelayCommand(Action execute, Func<bool> canExecute = null)
        {
            _execute = execute ?? throw new ArgumentNullException(nameof(execute));
            _canExecute = canExecute;
        }

        public bool CanExecute(object parameter) => _canExecute?.Invoke() ?? true;

        public void Execute(object parameter) => _execute();

        public event EventHandler CanExecuteChanged
        {
            add => CommandManager.RequerySuggested += value;
            remove => CommandManager.RequerySuggested -= value;
        }
    }

    public class RelayCommand<T> : ICommand
    {
        private readonly Action<T> _execute;
        private readonly Func<T, bool> _canExecute;

        public RelayCommand(Action<T> execute, Func<T, bool> canExecute = null)
        {
            _execute = execute ?? throw new ArgumentNullException(nameof(execute));
            _canExecute = canExecute;
        }
    }
}

```

```

    public bool CanExecute(object parameter) => _canExecute?.Invoke((T)parameter) ??
true;

    public void Execute(object parameter) => _execute((T)parameter);

    public event EventHandler CanExecuteChanged
    {
        add => CommandManager.RequerySuggested += value;
        remove => CommandManager.RequerySuggested -= value;
    }
}

```

Models.cs

```

using System;
using System.Collections.Generic;
using System.Text;

namespace PSListMaker.Models
{
    public class ComponentMin
    {
        public string Name { get; }

        public string Type { get; }

        public ComponentMin(string name, string type)
        {
            Name = name;
            Type = type;
        }
    }
}

```

ComponentsWithMult.cs

```

using System;
using System.Collections.Generic;
using System.Text;

namespace PSListMaker.Models
{
    public class ComponentsWithMult : ComponentMin
    {
        public ushort Multiplicity { get; }

        public ComponentMin? Parent { get; }
    }
}

```

```

        public string NameWithMultString
        {
            get
            {
                if (Multiplicity == 0)
                    return Name;

                return $"{Name} ({Multiplicity}x)";
            }
        }

        public ComponentsWithMult(string name, string type, ushort multiplicity, ComponentMin?
parent = null) : base(name, type)
        {
            Multiplicity = multiplicity;
            Parent = parent;
        }
    }
}

```

ComponentsWithSpecs.cs

```

using System;
using System.Collections.Generic;
using System.Text;

namespace PSListMaker.Models
{
    public class ComponentsWithSpecs : ComponentsWithMult
    {
        public List<ComponentsWithSpecs> Specs { get; }

        public ComponentsWithSpecs(string name, string type, ushort multiplicity,
ComponentMin parent = null) : base(name, type, multiplicity, parent)
        {
            Specs = new List<ComponentsWithSpecs>();
        }
    }
}

```

FileServiceWithActionHistory.cs

```

using ConsoleApp2.Entities;
using ConsoleApp2.MenuService;
using System;
using System.Collections.Generic;
using System.Drawing.Printing;

```

```

using System.IO;
using System.Linq;
using System.Text;
using System.Windows.Controls;
using System.Windows.Shell;

namespace PSListMaker.Models
{
    public class UndoableCommand
    {
        private EventHandler _undoCommand;

        public UndoableCommand(EventHandler undoCommand)
        {
            _undoCommand = undoCommand;
        }

        public void Undo()
        {
            _undoCommand?.Invoke(this, EventArgs.Empty);
        }
    }

    public interface IFileServiceWithActionHistory : IFileService
    {
        void DeleteTempFiles();

        bool IsCanUndo { get; }

        bool IsUnsave { get; }

        void Undo();

        void Save();

        void ClearUndoBuffer();
    }

    public class FileServiceWithActionHistory : FileService, IFileServiceWithActionHistory
    {
        private Stack<UndoableCommand> _commandBuffer = [];

        private TempSaver _tempSaver;

        private string _originProductPath;

        private string _originSpecsPath;

        public bool IsCanUndo => _commandBuffer.Count > 0;

        public bool IsUnsave { get; private set; } = false;
    }
}

```

```

public void ClearUndoBuffer()
{
    _commandBuffer.Clear();
}

public void Undo()
{
    if (_commandBuffer.Count == 1)
    {
        IsUnsave = false;
    }

    _commandBuffer.Pop().Undo();
}

public void Save()
{
    _commandBuffer.Clear();

    _productRepo.ChangeSpecFileName(Path.GetFileName(_originSpecsPath));

    _tempSaver.SaveToOrigin();

    _productRepo.ChangeSpecFileName(Path.GetFileName(_tempSaver.SpecTempPath));

    IsUnsave = false;
}

public void DeleteTempFiles()
{
    if (_tempSaver != null)
    {
        _tempSaver.DeleteTempFile();
    }
}

public override void Create(string directoryPath, string productFileName, string
specFileName, short dataLength)
{
    base.Create(directoryPath, productFileName, specFileName, dataLength);

    base.Close();

    Open(Path.Combine(directoryPath, $"{productFileName}.prd"));
}

public override void Open(string fullProductPath)
{

```

```

        if (_tempSaver != null)
        {
            _tempSaver.DeleteTempFile();
        }

        _tempSaver = new TempSaver(fullProductPath);
        _tempSaver.CreateTempFile();

        base.Open(_tempSaver.TempPath);

_productRepo.ChangeSpecFileName(Path.GetFileName(_tempSaver.SpecTempPath));

        base.Close();

        base.Open(_tempSaver.TempPath);

        _originProductPath = fullProductPath;

        _originSpecsPath = Path.Combine(Path.GetDirectoryName(fullProductPath),
            $"{Path.GetFileNameWithoutExtension(fullProductPath)}.prs");
    }

    public override void Close()
    {
        base.Close();
        if (_tempSaver != null)
        {
            _tempSaver.DeleteTempFile();
            _tempSaver = null;
        }
    }

    public override void Input(string componentName, ComponentType type)
    {
        IsUnsave = true;

        _commandBuffer.Push(new UndoableCommand((object? sender, EventArgs e) =>
        {
            List<string> deletedItems = GetDeletedComps();

            List<(string, string)> deletedSpecs = GetDeletedSpecs(deletedItems);

            base.Restore();

            base.Delete(componentName);

            base.Truncate();

            DeleteCompAndSpecs(deletedItems, deletedSpecs);
        }
    }

```

```

    ));

    base.Input(componentName, type);
}

public override void Input(string componentName, string specificationName)
{
    IsUnsave = true;

    _commandBuffer.Push(new UndoableCommand((object? sender, EventArgs e) =>
    {
        List<string> deletedItems = GetDeletedComps();

        List<(string, string)> deletedSpecs = GetDeletedSpecs(deletedItems);

        base.Restore();

        base.Delete(componentName, specificationName);

        base.Truncate();

        DeleteCompAndSpecs(deletedItems, deletedSpecs);
    }
    ));

    base.Input(componentName, specificationName);
}

public override void Input(string componentName, string specificationName, ushort
multiplicity)
{
    IsUnsave = true;

    _commandBuffer.Push(new UndoableCommand((object? sender, EventArgs e) =>
    {
        List<string> deletedItems = GetDeletedComps();

        List<(string, string)> deletedSpecs = GetDeletedSpecs(deletedItems);

        base.Restore();

        base.Delete(componentName, specificationName);

        base.Truncate();

        DeleteCompAndSpecs(deletedItems, deletedSpecs);
    }
    ));

    base.Input(componentName, specificationName, multiplicity);
}

```

```

public override void Delete(string componentName)
{
    IsUnsave = true;

    _commandBuffer.Push(new UndoableCommand((object? sender, EventArgs e) =>
    {
        base.Restore(componentName);
    }
    ));

    base.Delete(componentName);
}

public override void Delete(string componentName, string specificationName)
{
    IsUnsave = true;

    _commandBuffer.Push(new UndoableCommand((object? sender, EventArgs e) =>
    {
        ushort m = base.GetProductSpecifications(componentName).First(x => x.Name ==
specificationName).Multiplicity;

        base.Input(componentName, specificationName, m);
    }
    ));

    base.Delete(componentName, specificationName);
}

public override void Edit(string productName, string newProductName)
{
    IsUnsave = true;

    _commandBuffer.Push(new UndoableCommand((object? sender, EventArgs e) =>
    {
        base.Edit(newProductName, productName);
    }
    ));

    base.Edit(productName, newProductName);
}

public override void EditSpec(string productName, string specName, ushort
newMultiplicity)
{
    IsUnsave = true;

    _commandBuffer.Push(new UndoableCommand((object? sender, EventArgs e) =>
    {

```

```

        ushort oldM = base.GetProductSpecifications(productName).First(x => x.Name ==
specName).Multiplicity;

        base.EditSpec(productName, specName, oldM);
    }
});

base.EditSpec(productName, specName, newMultiplicity);
}

public override void Restore()
{
    IsUnsave = true;

    List<string> deletedItems = GetDeletedComps();
    List<(string, string)> deletedSpecs = GetDeletedSpecs(deletedItems);

    _commandBuffer.Push(new UndoableCommand((object? sender, EventArgs e) =>
    {
        DeleteCompAndSpecs(deletedItems, deletedSpecs);
    }
    ));

    base.Restore();
}

public override void Restore(string componentName)
{
    IsUnsave = true;

    _commandBuffer.Push(new UndoableCommand((object? sender, EventArgs e) =>
    {
        base.Delete(componentName);
    }
    ));

    base.Restore(componentName);
}

public override void Truncate()
{
    IsUnsave = true;

    List<(string, ComponentType)> deletedItems = GetDeletedCompsWithType();
    List<(string, string, ushort)> deletedSpecs =
GetDeletedSpecsWithMult(deletedItems.Select(x => x.Item1).ToList());

    _commandBuffer.Push(new UndoableCommand((object? sender, EventArgs e) =>
    {
        foreach (var item in deletedItems)
        {

```

```

        base.Input(item.Item1, item.Item2);
        base.Delete(item.Item1);
    }

    foreach (var item in deletedSpecs)
    {
        base.Input(item.Item1, item.Item2, item.Item3);
        base.Delete(item.Item1, item.Item2);
    }
}
));

base.Truncate();
}

private List<string> GetDeletedComps()
{
    return base.GetAllProducts()
        .Where(x => x.DelBit == 1)
        .Select(x => x.Name)
        .ToList();
}

private List<(string, ComponentType)> GetDeletedCompsWithType()
{
    return base.GetAllProducts()
        .Where(x => x.DelBit == 1)
        .Select(x => (x.Name, x.Type))
        .ToList();
}

private List<(string, string)> GetDeletedSpecs(List<string> deletedComps)
{
    List<(string, string)> deletedSpecs = [];

    foreach (var product in deletedComps)
    {
        foreach (var spec in GetProductSpecifications(product))
        {
            if (spec.IsDeleted)
            {
                deletedSpecs.Add((product, spec.Name));
            }
        }
    }

    return deletedSpecs;
}

private List<(string, string, ushort)> GetDeletedSpecsWithMult(List<string>
deletedComps)

```

```

    {
        List<(string, string, ushort)> deletedSpecs = [];

        foreach (var product in deletedComps)
        {
            foreach (var spec in GetProductSpecifications(product))
            {
                if (spec.IsDeleted)
                {
                    deletedSpecs.Add((product, spec.Name, spec.Multiplicity));
                }
            }
        }

        return deletedSpecs;
    }

    private void DeleteCompAndSpecs(List<string> deletedComps, List<(string, string)>
deletedSpecs)
    {
        foreach (var item in deletedComps)
        {
            base.Delete(item);
        }

        foreach (var item in deletedSpecs)
        {
            base.Delete(item.Item1, item.Item2);
        }
    }
}

```

TreeViewSearcher.cs

```

using PSListMaker.Models;
using System.Collections.Generic;
using System.Linq;
using System.Windows;
using System.Windows.Controls;

namespace PSListMaker.Searchers
{
    public class TreeViewSearcher
    {
        private readonly TreeView _treeView;
        private string _lastSearchText;
        private List<TreeViewItem> _currentMatches;
        private int _currentIndex = -1;
    }
}

```

```

public TreeViewSearcher(TreeView treeView)
{
    _treeView = treeView;
}

public void FindNext(string searchText)
{
    if (string.IsNullOrEmpty(searchText))
        return;

    if (_lastSearchText != searchText)
    {
        _lastSearchText = searchText;
        _currentMatches = GetAllMatchingTreeViewItems(searchText);
        _currentIndex = -1;
    }

    if (_currentMatches == null || _currentMatches.Count == 0)
    {
        MessageBox.Show("Совпадений не найдено.", "Поиск", MessageBoxButton.OK,
        MessageBoxImage.Information);
        return;
    }

    _currentIndex++;
    if (_currentIndex >= _currentMatches.Count)
    {
        MessageBox.Show("Достигнут конец списка. Начинаем сначала.", "Поиск",
        MessageBoxButton.OK, MessageBoxImage.Information);
        _currentIndex = 0;
    }

    // Выделяем найденный элемент
    SelectItem(_currentMatches[_currentIndex]);
}

private List<TreeViewItem> GetAllMatchingTreeViewItems(string searchText)
{
    var allItems = GetAllTreeViewItems(_treeView);
    return allItems
        .Where(item => (item.Header as ComponentMin)?.Name.IndexOf(searchText,
        StringComparison.OrdinalIgnoreCase) >= 0)
        .ToList();
}

private List<TreeViewItem> GetAllTreeViewItems(TreeView treeView)
{
    var result = new List<TreeViewItem>();

    foreach (var item in treeView.Items)
    {

```

```

        var container = treeView.ItemContainerGenerator.ContainerFromItem(item) as
TreeViewItem;
        if (container != null)
        {
            TraverseTreeViewItem(container, result);
        }
    }

    return result;
}

private void TraverseTreeViewItem(TreeViewItem item, List<TreeViewItem> list)
{
    list.Add(item);

    bool wasExpanded = item.IsExpanded;
    if (!item.IsExpanded)
        item.IsExpanded = true;

    item.UpdateLayout();

    foreach (var child in item.Items)
    {
        var childContainer = item.ItemContainerGenerator.ContainerFromItem(child) as
TreeViewItem;
        if (childContainer != null)
        {
            TraverseTreeViewItem(childContainer, list);
        }
    }
}

private void SelectItem(TreeViewItem item)
{
    // Снимаем выделение с предыдущего элемента
    if (_currentMatches != null)
    {
        foreach (var oldItem in _currentMatches)
        {
            if (oldItem != item)
                oldItem.IsSelected = false;
        }
    }

    item.IsSelected = true;
    item.BringIntoView();
    item.Focus();
}

public void Reset()
{

```

```
        _lastSearchText = null;
        _currentMatches = null;
        _currentIndex = -1;
    }
}
```

AddComponentWindowViewModel.cs

```
using ConsoleApp2.Entities;
using ConsoleApp2.MenuService;
using System;
using System.Collections.Generic;
using System.Runtime.Serialization.Formatters;
using System.Security.Cryptography;
using System.Text;

namespace PSListMaker.ViewModels
{
    public class AddComponentWindowViewModel
    {
        private IFileService _fileService;

        public AddComponentWindowViewModel(IFileService fileService)
        {
            _fileService = fileService;
        }

        public void Add(string name, ComponentType type)
        {
            _fileService.Input(name, type);
        }

        public void RegisterErrorHandler(EventHandler<string> errorHandler)
        {
            _fileService.ErrorOccurred += errorHandler;
        }

        public void UnRegisterErrorHandler(EventHandler<string> errorHandler)
        {
            _fileService.ErrorOccurred -= errorHandler;
        }
    }
}
```

AddSpecificationWindowViewModel.cs

```
using ConsoleApp2.MenuService;
using System;
```

```

using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace PSListMaker.ViewModels
{
    public class AddSpecificationWindowViewModel
    {
        private IFileService _fileService;

        private readonly string _componentName;

        public AddSpecificationWindowViewModel(string componentName, IFileService
fileService)
        {
            _fileService = fileService;
            _componentName = componentName;
        }

        public List<string> GetAllProductNames()
        {
            var allNames = _fileService.GetAllProducts()
                .Where(x => x.Type != ConsoleApp2.Entities.ComponentType.Product)
                .Select(x => x.Name)
                .ToList();

            var existingSpecNames = new HashSet<string>(
                _fileService.GetProductSpecifications(_componentName)
                    .Select(p => p.Name));

            return allNames.Where(n => !existingSpecNames.Contains(n)).ToList();
        }

        public void AddSpecification(string specification, ushort multiplicity = 1)
        {
            _fileService.Input(_componentName, specification, multiplicity);
        }

        public void RegisterOnChange(EventHandler e)
        {
            _fileService.ProductsChanged += e;
        }

        public void UnRegisterOnChange(EventHandler e)
        {
            _fileService.ProductsChanged -= e;
        }
    }
}

```

ChangeMultiplicityWindowViewModel.cs

```
using ConsoleApp2.MenuService;
using System;
using System.Collections.Generic;
using System.Net;
using System.Text;

namespace PSListMaker.ViewModels
{
    public class ChangeMultiplicityWindowViewModel
    {
        private IFileService _fileService;

        private string _component;

        private string _specs;

        public ChangeMultiplicityWindowViewModel(string component, string specs, IFileService fileService)
        {
            _fileService = fileService;
            _component = component;
            _specs = specs;
        }

        public void ChangeMultiplicity(ushort newMultiplicity)
        {
            _fileService.EditSpec(_component, _specs, newMultiplicity);
        }
    }
}
```

ComponentsListViewModel.cs

```
using ConsoleApp2.MenuService;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Text;

using PSListMaker.Models;
using System.Windows.Input;

namespace PSListMaker.ViewModels
{
    public class ComponentsListViewModel
    {

```

```

private IFileServiceWithActionHistory _fileService;

public bool IsUnsave => _fileService.IsUnsave;

public bool IsCanUndo => _fileService.IsCanUndo;

public ComponentsListViewModel(IFileServiceWithActionHistory fileService)
{
    _fileService = fileService;
}

public List<ComponentMin> GetComponents()
{
    List<ComponentMin> components = new List<ComponentMin>();

    foreach(var comp in _fileService.GetAllProducts())
    {
        components.Add(new ComponentMin(comp.Name, comp.Type.ToString()));
    }

    return components;
}

public void DeleteComponent(string name)
{
    _fileService.Delete(name);
}

public void Save()
{
    _fileService.Truncate();
    _fileService.Save();
}

public void Undo()
{
    _fileService.Undo();
}

public void RegisterOnChange(EventHandler update)
{
    _fileService.ProductsChanged += update;
}

public void UnRegisterOnChange(EventHandler update)
{
    _fileService.ProductsChanged -= update;
}
}

```

EditComponentWindowViewModel.cs

```
using ConsoleApp2.MenuService;
using System;
using System.Collections.Generic;
using System.Text;

namespace PSListMaker.ViewModels
{
    public class EditComponentWindowViewModel
    {
        private readonly IFileService _fileService;

        public EditComponentWindowViewModel(IFileService fileService)
        {
            _fileService = fileService;
        }

        public void EditComponent(string productName, string newProductName)
        {
            _fileService.Edit(productName, newProductName);
        }
    }
}
```

MainWindowViewModel.cs

```
using ConsoleApp2.MenuService;
using PSListMaker.Commands;
using PSListMaker.Models;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.IO;
using System.Text;
using System.Windows;
using System.Windows.Input;

namespace PSListMaker.ViewModels
{
    public class MainWindowViewModel
    {
        private readonly IFileServiceWithActionHistory _fileService;

        public MainWindowViewModel(IFileServiceWithActionHistory fileService)
        {
            _fileService = fileService;
        }
    }
}
```

```

    public void CreateFile(string path, short dataLength)
    {
        string directory = Path.GetDirectoryName(path);
        string fileNames = Path.GetFileNameWithoutExtension(path);

        _fileService.Create(directory, fileNames, fileNames, dataLength);
    }

    public void OpenFile(string path)
    {
        _fileService.Open(path);
    }

    public void DeleteTempFiles()
    {
        _fileService.Close();
        _fileService.DeleteTempFiles();
    }

    public void RegisterOnError(EventHandler<string> handler)
    {
        _fileService.ErrorOccurred += handler;
    }

    public void UnRegisterOnError(EventHandler<string> handler)
    {
        _fileService.ErrorOccurred -= handler;
    }
}

```

SpecificationViewModel.cs

```

using ConsoleApp2.Entities;
using ConsoleApp2.MenuService;
using PSListMaker.Models;
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.Data;
using System.Runtime.InteropServices;
using System.Text;

namespace PSListMaker.ViewModels
{
    public class SpecificationsViewModel
    {
        private IFileService _fileService;
    }
}

```

```

public SpecificationsViewModel(IFileService fileService)
{
    _fileService = fileService;

    Components = new(GetComponentsTree());

    _fileService.ProductsChanged += UpdateProperties;
}

public ObservableCollection<ComponentsWithSpecs> Components { get; private set; }

public List<ComponentsWithSpecs> GetComponentsTree()
{
    var componentsWithSpecs = new List<ComponentsWithSpecs>();

    var components = _fileService.GetAllProducts();

    foreach (var component in components)
    {
        if (component.Type == ComponentType.Product)
        {
            var componentsWithSpec = new ComponentsWithSpecs(component.Name,
                component.Type.ToString(), 0);

            AddSpecifications(componentsWithSpec);

            componentsWithSpecs.Add(componentsWithSpec);
        }
    }

    return componentsWithSpecs;
}

private void AddSpecifications(ComponentsWithSpecs component)
{
    var specs = _fileService.GetProductSpecifications(component.Name);

    if (specs != null)
    {
        foreach (var spec in specs)
        {
            var child = new ComponentsWithSpecs(spec.Name,
                _fileService.GetProduct(spec.Name).Type.ToString(), spec.Multiplicity,
                component);

            AddSpecifications(child);

            component.Specs.Add(child);
        }
    }
}

```

```

    }

    public void AddSpecs(string componentName, string specificationName, ushort
multiplicity)
    {
        _fileService.Input(componentName, specificationName, multiplicity);
    }

    public void RemoveSpecs(string componentName, string specificationName)
    {
        _fileService.Delete(componentName, specificationName);
    }

    public void EditSpecs(string componentName, string specificationName, ushort
newMultiplicity)
    {
        _fileService.EditSpec(componentName, specificationName, newMultiplicity);
    }

    private void UpdateProperties(object? sender, EventArgs e)
    {
        Components.Clear();

        foreach (var component in GetComponentsTree())
        {
            Components.Add(component);
        }
    }

    public void UnRegister()
    {
        _fileService.ProductsChanged -= UpdateProperties;
    }
}

```

ComponentListService.cs

```

using ConsoleApp2.MenuService;
using PSListMaker.Views;
using System;
using System.Collections.Generic;
using System.Text;

namespace PSListMaker.WindowServices
{
    public class ComponentListService : IComponentListService
    {
        private IFileService _fileService;
    }
}

```

```

    public ComponentListService(IFileService fileService)
    {
        _fileService = fileService;
    }

    public AddComponentWindow GetAddWindow()
    {
        return new AddComponentWindow(new
ViewModels.AddComponentWindowViewModel(_fileService));
    }

    public EditComponentWindow GetEditWindow(string oldComponentName)
    {
        return new EditComponentWindow(oldComponentName, new
ViewModels.EditComponentWindowViewModel(_fileService));
    }
}

```

IComponentListService.cs

```

using PSListMaker.Views;
using System;
using System.Collections.Generic;
using System.Text;

namespace PSListMaker.WindowServices
{
    public interface IComponentListService
    {
        AddComponentWindow GetAddWindow();

        EditComponentWindow GetEditWindow(string oldComponentName);
    }
}

```

IMainWindowService.cs

```

using System;
using System.Collections.Generic;
using System.Text;

namespace PSListMaker.WindowServices
{
    public interface IMainWindowService
    {
        ComponentsList GetComponentsListWindow();

        Specifications GetSpecificationsWindow();
    }
}

```

```
}  
}
```

MainWindowService.cs

```
using ConsoleApp2.MenuService;  
using PSListMaker.Models;  
using System;  
using System.Collections.Generic;  
using System.Reflection.Metadata;  
using System.Text;  
  
namespace PSListMaker.WindowServices  
{  
    public class MainWindowService : IMainWindowService  
    {  
        private IFileServiceWithActionHistory _fileService;  
  
        public MainWindowService(IFileServiceWithActionHistory fileService)  
        {  
            _fileService = fileService;  
        }  
  
        public ComponentsList GetComponentListWindow()  
        {  
            return new ComponentsList(new ComponentListService(_fileService), new  
ViewModels.ComponentsListViewModel(_fileService));  
        }  
  
        public Specifications GetSpecificationsWindow()  
        {  
            return new Specifications(new SpecificationWindowService(_fileService), new  
ViewModels.SpecificationsViewModel(_fileService));  
        }  
    }  
}
```

SpecificationWindowService.cs

```
using ConsoleApp2.MenuService;  
using PSListMaker.Views;  
using System;  
using System.Collections.Generic;  
using System.Text;  
  
namespace PSListMaker.WindowServices  
{  
    public class SpecificationWindowService : ISpecificationWindowService  
    {  

```

```

private readonly IFileService _fileService;

public SpecificationWindowService(IFileService fileService)
{
    _fileService = fileService;
}

public AddSpecificationWindow GetAddWindow(string componentName)
{
    return new AddSpecificationWindow(componentName,
        new ViewModels.AddSpecificationWindowViewModel(componentName,
            _fileService));
}

public ChangeMultiplicityWindow GetChangeMultiplicityWindow(string componentName,
    string specName)
{
    return new ChangeMultiplicityWindow(
        new ViewModels.ChangeMultiplicityWindowViewModel(componentName,
            specName, _fileService));
}
}

```

AssemblyInfo.cs

```

using System.Windows;

[assembly: ThemeInfo(
    ResourceDictionaryLocation.None,          //where theme specific resource dictionaries are
    located
                                         //(used if a resource is not found in the page,
                                         // or application resource dictionaries)
    ResourceDictionaryLocation.SourceAssembly //where the generic resource dictionary is
    located
                                         //(used if a resource is not found in the page,
                                         // app, or any theme specific resource dictionaries)
)]

```

4. Скриншоты дампа файлов

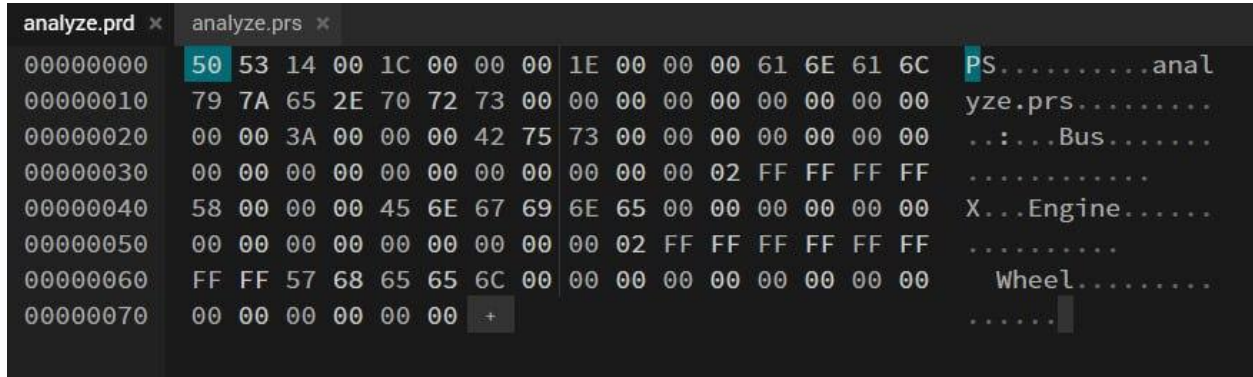


Рис. 1 – analyze.prd

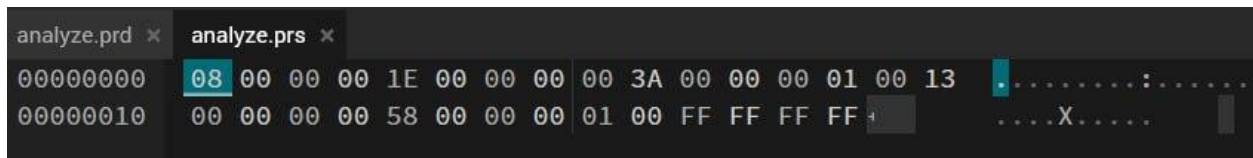


Рис. 2 – analyze.prs

Краткое описание дампа файлов

PRD файл:

Заголовок:

0x0 (2) Сигнатура PS

0x2 (2) DataLength 20

0x4 (4) FirstRecPtr 28

0x8 (4) UnclaimedPtr

0xC (20) SpecFileName analyze.prs

Запись 1 смещение 0x1C (28):

0x1C (1) DelBit 0

0x1D (1) Type Product

0x1E (4) Указатель на Spec

0x22 (4) NextProductPtr 58

0x26 (20) Имя Bus

Запись 2 смещение 0x3A (58):

0x3A (1) DelBit 0

0x3B (1) Type Detail

0x3C (4) Указатель на Spec

0x40 (4) NextProductPtr 88

0x44 (20) Имя Engine

Запись 3 смещение 0x58 (88):

0x58 (1) DelBit 0

0x59 (1) Type Detail

0x5A (4) Указатель на Spec

0x5E (4) NextProductPtr -

0x62 (20) Имя Wheel

PRS файл:

Заголовок:

0x0 (4) FirstRecPtr 8

0x4 (4) UnclaimedPtr 30

Запись 1 смещение 0x8 (8):

0x8 (1) Бит удаления 0

0x9 (4) Компонент 58

0xD (2) Кратность 1

0xF (4) Следующая спецификация 19

Запись 2 смещение 0x13 (19):

0x13 (1) Бит удаления 0

0x14 (4) Компонент 88

0x18 (2) Кратность 1

0x1A (4) Следующая спецификация –

5. Ссылка на github репозиторий проекта:

<https://github.com/mrrred/Lab1Sem4/>